



Media Engineering and Technology Faculty
German University in Cairo

The Logic Advocate Tutor

Bachelor Thesis

Author: Adel Ashraf Mohamed Kamel Emam

Supervisors: Dr. Ahmed M. H. ABDELFAHATTAH

Submission Date: 8 June 2026



Media Engineering and Technology Faculty
German University in Cairo

The Logic Advocate Tutor

Bachelor Thesis

Author: Adel Ashraf Mohamed Kamel Emam
Supervisors: Dr. Ahmed M. H. ABDELFATTAH
Submission Date: 8 June 2026

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor's Degree
- (ii) due acknowledgment has been made in the text to all other material used

Adel Ashraf Mohamed Kamel Emam
8 June 2026

Acknowledgments

I want to thank my supervisor, Dr. Ahmed M. H. Abdelfattah, whose guidance throughout this project shaped both the formal direction of my work and the day to day decisions that turned a research idea into something I could actually run on a laptop. His balance of academic rigour and engineering pragmatism set the tone for my whole thesis.

I am grateful to the Faculty of Media Engineering and Technology at the German University in Cairo for the resources and the scholarly environment in which I carried out my work. I am also grateful to the broader argumentation theory community, whose decades of careful work on Dung style frameworks, bipolar extensions, gradual semantics, and dialogue games provided every formal idea I built on.

A special thank you goes to the classmates who volunteered their time to test my system over informal sessions. Their feedback shaped the user facing parts of my design more than they probably realise. I hope debating an AI was at least mildly entertaining.

Finally, I thank my family for their unwavering patience and support throughout my writing process. None of this would have been possible without them.

Abstract

I see critical thinking as one of the things higher education is supposed to teach, but I find that the tools we have to practise it on a screen all suffer from one of two opposite problems. Generative Large Language Models can hold a fluent debate on almost any topic, yet they exhibit *rhetorical rigidity*. Once a model is assigned a stance, it tends to defend flawed reasoning with confidence rather than admit that a premise has been defeated. Formal argumentation frameworks have the opposite weakness. They can decide who is winning a debate with mathematical certainty, but they offer no natural conversational interface, and they reduce a learner’s feedback to a flat accept or reject verdict. I find that neither half is a complete tutor on its own.

In this thesis I introduce my *Logic Advocate Tutor*, an interactive web application that brings the two halves together. My idea is to let the language model write the prose while a separate, deterministic engine writes the verdict. My engine is a custom Python evaluator that extends Dung’s abstract argumentation framework with a parallel support relation, continuous acceptability scores in $[0, 1]$ in place of binary pass or fail labels, integer per argument weights, and four value tags (Fact, Logic, Ethics, Emotion) borrowed from Bench-Capon’s value based audience model. On classical inputs my evaluator reduces to a thresholded grounded extension. It only departs from that binary world when bipolar, weighted, or value tagged inputs ask for something more nuanced.

My conversational half runs on LLaMA-3 70B through the Groq inference endpoint. I made the model talk back through a strict pipe separated response protocol, and I never allow it to decide who is winning the debate. Every score on the screen comes from my engine, not from the LLM. So the AI cannot pretend it is winning when the math says it has lost. When my engine declares the AI’s argument defeated, the AI emits a clean concession sentinel and the debate ends. I handle voice input through OpenAI’s Whisper, image evidence through the LLaMA-3.2 90B Vision model, and online multiplayer through an ngrok tunnel and a QR code that lets two students on different networks share a debate in seconds. The whole thing runs inside Streamlit and stays within Groq’s free tier, so a student can use it without a credit card.

A formal controlled user study was outside the time budget of a one semester Bachelor project. I instead validated my system through structural inspection of the engine, informal play sessions with classmates, and constant observation while building it. My engine appeared to behave the way the formula predicts. My AI Layer stayed consistent with the engine’s verdict in every session I watched. Latency was low enough that nobody complained. And in the informal feedback I collected, the moment the AI conceded was

the single most cited reaction. I identify a proper study with a baseline condition and a learning outcome measurement as my highest priority follow up work. I offer the thesis itself as evidence that the right way to build educational AI in the era of large language models is to keep the model and the rules separate, and to let the rules win whenever they disagree.

Contents

List of Figures

Chapter 1

Introduction

1.1 The Illusion of Competence in Educational AI

I have watched the integration of Artificial Intelligence (AI) into education transform several aspects of how I learn. In the past, automated tutors worked exceptionally well for objective subjects like math or programming. In those fields, there is a clear right or wrong answer, and student responses can be graded with strict rule based systems. But I noticed those traditional tutors hit a wall when they try to support subjective decision making tasks [?]. In subjects that require critical thinking, ethics, or nuanced debate, conclusions are rarely simple. They depend on context, on weighing conflicting perspectives, and on understanding human values [?].

When I deal with these grey areas, I naturally fall back on argumentation. By argumentation I mean a back and forth dialogue grounded in defeasible reasoning [?]. By defeasible reasoning I mean my conclusions are drawn from premises that are presumed but not deductively guaranteed. If a premise is successfully challenged by new and stronger evidence, the argument is defeated. Modern education leans heavily on this interactive process to develop students' critical thinking skills.

While human to human debate remains the best way to practise these skills, I see Large Language Models (LLMs) emerging as powerful educational tools. In subjective tasks, an AI agent can act as a tireless devil's advocate [?]. It can offer different perspectives to stimulate a student's reasoning and expose them to viewpoints they might have overlooked. But to capture the fluid nature of human debate inside a computer system, I need a rigid structural backbone. In computer science, I get that backbone from Computational Argumentation. The field is rooted in Dung's Abstract Argumentation Framework, which gives me a mathematical way to determine whether an argument survives based on how it attacks and defends other arguments [?].

1.2 The Problem of Rhetorical Rigidity

Despite the educational promise of AI, I quickly hit a serious problem the moment I tried to use off the shelf language models as standalone logic tutors. They suffer from an illusion of competence. Generative models are very good at producing human sounding text, but I found that they often fail to capture the pragmatic reasoning required to surface implicit premises [?]. Those unstated assumptions are what actually make an argument persuasive in the first place.

Worse, when these models are tasked with arguing a specific viewpoint, I observed that they are highly susceptible to *rhetorical rigidity* [?]. By rhetorical rigidity I mean the tendency of an LLM, once assigned a stance, to keep making confident assertions even after the underlying logic has been defeated. The literature confirms this. When an AI is forced to defend a rigidly assigned position, it tends to overcommit to flawed reasoning [?]. It adopts an artificially assertive tone instead of objectively evaluating the logical weight of the user’s counter argument. In a classroom setting, I see this as disastrous. A student might present a perfectly valid and game winning logical point, only for the AI to hallucinate facts or stubbornly double down on a defeated premise. The lesson the student learns is that volume and stubbornness outweigh logical soundness. That is the opposite of what I want a tutor to teach.

Traditional logic tutors crafted by human experts go in the other direction. I find them mathematically flawless but rigid and time consuming to build. They struggle to adapt to the messy and open ended nature of natural language. They cannot easily parse the subtle assumptions human debaters rely on. So I am left with a real gap in the current landscape. On one side, I have formal logic solvers that are mathematically accurate but conversationally dead. On the other, I have modern language models that are conversationally brilliant but logically unreliable.

1.3 Constraining the Ghost: The Logic Advocate Tutor

To bridge this gap, I introduce the **Logic Advocate Tutor**. I built it as a real time interactive tutoring system that helps students practise structured and logically grounded argumentation. I designed the system around a novel premise. It synthesises the conversational fluidity of state of the art generative AI with the mathematical chains of Computational Argumentation. In essence, I constrain the language model inside a strict mathematical graph.

At its core, my logic engine upgrades Dung’s standard Abstract Argumentation Framework to support Bipolar Argumentation and Gradual Evaluation Semantics. Instead of letting the AI guess who is winning the debate, I translate every natural language input into a visual, directed graph of supports and attacks. I compute a continuous acceptability score in the range $[0.0, 1.0]$ for every argument on the fly. I then inject the state of that

mathematical graph directly into the system prompt, forcing the AI to stay consistent with the objective state of the debate. The model cannot claim victory if the math says its argument has been defeated.

To keep the system accessible and responsive in real time, I use the zero cost, high speed Groq API to power Meta’s LLaMA 3 models, paired with OpenAI’s Whisper for voice transcription. I also implemented robust session management, including a custom manual save database and dynamic visual logic maps. Together they create a persistent educational arena.

1.4 Thesis Contributions

My contributions span both theoretical implementation and practical software engineering. They are:

1. **A custom Bipolar Gradual logic engine.** I designed and implemented a Python evaluator that translates natural language arguments into a weighted bipolar argumentation framework. The engine combines bipolarity, gradual semantics, value based weighting, and preference based normalisation, and computes continuous acceptability scores in $[0.0, 1.0]$.
2. **A protocol for grounding the LLM in the engine’s verdict.** I built a structured pipe separated response format and context injection scheme that, by construction, forbids the LLM from computing the verdict itself. Every score on the screen comes from my deterministic engine. I designed this to eliminate hallucination about debate outcomes and to mitigate the rhetorical rigidity problem identified by recent work on multi agent LLM debate [?].
3. **An on demand automated hint system.** Drawing inspiration from data driven hint factory methodologies [?], I made the tutor surface a single sentence strategic hint targeted at the most recent enemy argument whenever the learner asks for one. I made sure the hint does not consume the learner’s turn, which keeps the help non evaluative.
4. **Interactive visualisation and persistence.** I integrated Graphviz to render real time, colour coded maps of the ongoing debate, and I added a horizontal momentum bar that summarises which side is currently ahead. I also wrote a custom JSON based save format that lets students preserve, load, and review past debates.
5. **A multimodal, classroom deployable architecture.** I delivered the whole system as a low latency Streamlit web application, with Whisper powered voice input, image evidence handled by the LLaMA-3.2 90B Vision model, and ngrok based QR code multiplayer joining for online debates between students on different networks.

I examine the system’s behaviour informally in this thesis through structural inspection of the engine and small scale play sessions with classmates. I identify a controlled study with a baseline condition and learning outcome measurement in Chapter ?? as my highest priority follow up work.

1.5 Roadmap of the Thesis

I trace the development of the Logic Advocate Tutor from its theoretical roots to its final technical execution.

In Chapter 2, I lay out the background and the theoretical groundwork. I cover Dung’s Abstract Argumentation Framework, the evolution of bipolar and gradual semantics, the formal dialogue games that translate static graphs into interactive conversations, and the current limitations of language models in subjective decision making.

In Chapter 3, I describe my methodology. I walk through the four layer architecture of the system, the formal definition of my Bipolar Gradual logic engine, the dialogue game protocol that governs every move, and the pedagogical affordances that bring the engine in front of a learner.

In Chapter 4, I dive into the implementation. I walk through the Python that realises the design, paying close attention to the prompt engineering protocol I built to discipline the LLM, the polled JSON architecture of the multiplayer subsystem, and the engineering trade offs I had to make along the way.

In Chapter 5, I look at how the system actually performs. Within the time budget of a one semester project, I did not run a formal user study. Instead, I report structural observations about the engine, informal feedback from classmates who used the tutor, and a discussion of what a proper evaluation would require.

In Chapter 6, I conclude the thesis. I summarise my contributions, acknowledge the system’s limitations, and lay out a concrete forward roadmap of future work in educational AI tools.

Chapter 2

Background and Literature Review

2.1 Introduction

My Logic Advocate Tutor does not exist in a vacuum. I built it at the intersection of three fields. Formal computational logic, modern artificial intelligence, and educational pedagogy. To explain how my system works, I have to trace how computer scientists turned human debate into mathematical graphs, how Large Language Models (LLMs) interact with those graphs, and how intelligent tutoring systems use data to keep students from giving up.

2.2 The Bedrock of Logic: Abstract Argumentation Frameworks

I trace the mathematical foundation of modern computational argumentation back to Phan Minh Dung’s seminal 1995 paper [?]. Before Dung, evaluating an argument meant parsing its internal text and its logical proofs. I credit Dung with revolutionising the field by introducing the Abstract Argumentation Framework (AF), which zooms out and looks at the relationships between arguments instead [?].

In Dung’s framework, I treat arguments as abstract nodes, and I place the focus entirely on the binary “attack” relationships between them [?]. In this calculus of opposition, an argument’s survival is not based on its inherent truth, but on its ability to defend itself against counter arguments. To determine which arguments “win” a debate, Dung introduced acceptability semantics like the Grounded Extension, which mathematically calculate the set of arguments that remain undefeated after all attacks are resolved [?]. Subsequent work has consolidated and extended these semantics into a broad family. I find the modern handbook chapter by Baroni, Caminada, and Giacomin [?] a useful starting point for any reader who wants the formal landscape in one place.

2.3 Beyond Binary: Bipolar and Gradual Frameworks

While I find Dung’s original framework great for modelling negative interactions, real human debate is rarely purely adversarial. Participants build on each other’s points, agree, and strengthen each other’s claims. To capture that reality, researchers introduced Bipolar Argumentation Frameworks (BAFs), which add a secondary “support” relation to the graph [?]. In a BAF, an argument can directly reinforce the acceptability of its target, simulating a more collaborative debate environment [?].

Traditional logic also evaluates arguments in a strict binary fashion. An argument is either accepted (IN) or defeated (OUT) [?]. I prefer the modern alternative. Gradual Evaluation Semantics assign continuous acceptability scores in $[0.0, 1.0]$ [?]. Under gradual semantics, an argument is not destroyed by a single attack. Instead its logical weight is proportionally diminished [?]. Besnard and Hunter’s h categorizer is a concrete and influential instantiation of this idea, scoring an argument by $1/(1 + \sum_{b \rightarrow a} s(b))$ [?]. I read several of the more recent gradual semantics as generalisations of it. Value Based and Preference Based Argumentation Frameworks expand on the same theme by assigning numeric weights to specific categories of arguments, so that the math can account for the fact that a weaker argument cannot freely defeat a stronger one [?].

2.4 Structuring the Conversation: Dialogue Games

To turn these static mathematical graphs into interactive chat applications, I lean on the literature on “Dialogue Games” [?]. Dialogue games are rule governed interactions in which two or more parties take turns making specific utterances such as claims, challenges, concessions, or retractions [?]. The goal is to persuade the opponent or resolve a conflict of opinion.

When I force a conversation to follow these rules, the interaction implicitly builds a logical graph of arguments and counter arguments over time [?]. I see the integration of formal dialogue structures with natural language processing as the foundation for Argumentation Based Chatbots [?]. Where standard chatbots rely on simple pattern matching or pure generative guessing, argumentation based agents map user inputs to a formal knowledge base, which lets them engage in more strategic persuasion and deliberation. Castagna et al. [?] survey several deployed systems in this family, including persuasion dialogue systems for medical decision making and computational mediators that propose compromises in multi party negotiation. The common thread across all of them is the same separation of structure (the formal graph) from surface (the generated text) that I adopt in my own thesis.

2.5 The AI Frontier: LLMs and Subjective Decision Making

The advent of Large Language Models has dramatically changed how I see conversational agents operating. In subjective decision making tasks, where there is no single objective “ground truth,” I see LLMs increasingly used to present users with divergent rationales and to act as a collaborative “devil’s advocate” [?].

But I find relying entirely on LLMs in these environments risky. In isolation, LLMs frequently struggle to grasp “implicit premises”. Those are the unstated, pragmatic assumptions that bridge a claim and its reasoning [?]. When an LLM is assigned a strict adversarial stance in a debate, it often exhibits “rhetorical rigidity” [?]. Instead of objectively evaluating the logic of an opponent’s argument, the model overcommits to its assigned stance, defending flawed reasoning with unwarranted confidence and extreme tones [?]. To address this, researchers have suggested that embedding a computational argumentation engine inside the generative loop can mitigate these weaknesses by anchoring the model’s text generation to a verifiable, mathematical state [?].

I read the literature so far as diagnosing the rhetorical rigidity problem and gesturing at this kind of fix at the conceptual level. What I add in this thesis is a concrete, deployable instance of that fix. An undergraduate targeted tutor in which the LLM is genuinely unable to compute the verdict of a debate, because I have wrested that computation away from it and handed it to a separate engine.

2.6 Educational Technology and Automated Hint Generation

In the educational sector, I see argumentation recognised as a fundamental instrument for developing critical thinking and collaborative problem solving skills [?]. I also see that building Intelligent Tutoring Systems (ITS) that can effectively guide students through complex logic problems is notoriously difficult, because it traditionally requires immense time from subject matter experts to hand craft every possible rule and response [?].

To get past this bottleneck, researchers developed data driven approaches like the “Hint Factory” to automate the generation of formative feedback [?]. By mapping out historical student interactions as Markov Decision Processes, these systems can automatically derive context specific hints based on optimal solution paths [?]. Experimental evaluations in deductive logic courses show that students provided with on demand, automated hints attempt and complete significantly more problems [?]. They are also substantially less likely to abandon the tutor out of frustration [?].

I draw on the on demand and non evaluative principles of the Hint Factory [?], but I implement my hint generator as a single LLM call rather than as an MDP based search over historical interactions. The technical mechanism I use is therefore different from Stamper et al.’s system, but the pedagogical philosophy I follow is the same. Give help when asked, and do not penalise the request.

2.7 Summary

I read the literature as showing a clear trajectory. Formal argumentation provides flawless mathematical tracking of debates [?], and LLMs provide unparalleled conversational fluidity [?]. Yet I find neither of them sufficient as a standalone educational tool. LLMs suffer from rhetorical rigidity [?], and abstract graphs lack natural human interaction. To keep students engaged, I also need dynamic, non evaluative hint systems [?]. In my Logic Advocate Tutor I synthesise these strands of research, using my Bipolar Gradual logic engine to constrain a generative LLM inside a formal dialogue game. The result is a robust, interactive environment for teaching argumentation.

Chapter 3

Methodology

3.1 Overview

In this chapter I walk through how I designed the Logic Advocate Tutor before writing a single line of production code. I cover the abstractions I built to bridge formal argumentation theory and an actual chat style interface, and the reasoning I used to pick one approach over another. In the next chapter I show the Python that brought all of this to life.

I can sum up my core problem in one sentence. I needed a debate environment where any natural language argument is forced to follow the same survival rules that a formal argumentation framework would apply, while still feeling responsive enough to behave like a normal chat app. To pull this off, I had to make the system do four jobs at once. It accepts free text arguments from a human or a Large Language Model. It maps those arguments into a directed graph of attacks and supports. It computes a real time acceptability score for every node. And it shows the result of that score back to the participants in a way that actually changes how they play their next move.

That is why I broke the system into four cooperating layers. I built a **Presentation Layer** for the UI and the visual feedback, a **Dialogue Layer** that enforces the rules of the game, a **Logic Layer** that holds the framework and runs the gradual semantics, and an **AI Layer** that talks to the LLM and turns its replies back into valid argument moves. Figure 3.1 shows how the layers depend on each other. Each layer only knows about the layer below it. I made that isolation deliberate so I could swap out the language model, or rip out the UI, without breaking the math.

3.2 Design Goals and Non Goals

Before I get to the architecture, I want to be upfront about which problems my system actually tries to solve, and which I left for someone else to deal with later. My goals all came out of the gaps I identified back in Chapter 2.

Goal 1: Keep the LLM honest. I noticed that a standalone LLM debate agent suffers from rhetorical rigidity [?] and tends to claim victory no matter what is actually happening in the debate [?]. So I needed my system to maintain its own ground truth view of the debate, computed by an algorithm I trust rather than by a model that sometimes lies for the sake of looking confident. I let the LLM produce text and score how persuasive an argument is, but I do not let it decide who is winning.

Goal 2: Continuous scores, not pass or fail labels. Classical Dung style semantics declare every argument either **IN** or **OUT** [?]. I find that binary verdict fine for theory but useless for a learner. If a student’s claim is just told it has been “defeated”, they have no idea how close it was, what to fix, or whether one more support would have saved it. Following the gradual semantics literature [?, ?], I gave every argument a continuous score between 0 and 1 so the learner can watch their argument get weaker rather than just being told it died.

Goal 3: Make support a first class citizen. I see real classroom debate as collaborative, not just adversarial. People build on each other’s points all the time. So I extended the basic attack relation with a parallel *support* relation, the way bipolar argumentation handles it [?].

Goal 4: Different audiences value different kinds of arguments. In my view, a factual claim should hit harder against an emotional one than the other way around, at least if the audience is fact leaning. Following Bench-Capon’s value based framework [?], I added a value tag to every argument, and the tag scales the strength of attacks and supports across tags.

Goal 5: Run anywhere, cost nothing. I needed my system to live inside a normal browser, take both text and voice, and let a student fire it up without any installation step. I also needed it to fit inside the free tier of every cloud service it touches, so a student can use it without a credit card.

What I am not trying to do. My system does not do automatic argument mining from long essays [?]. It does not detect informal fallacies in your text. And it does not do the deep parsing I would need to extract Toulmin style warrants [?]. I treat each argument as a black box node, the way Dung treats them [?]. The framework does not look inside. The interesting bit is how I score those nodes, display them, and combine them with an LLM, not how their text gets parsed.

3.3 The Four Layer Architecture

Figure 3.1 shows the layers I built and what flows between them. A user move (text, voice, or an image) enters at the top and gets pushed downward. Recomputed scores and the AI’s reply travel back up to the screen. The figure colours **SYMBOLIC** components in gold (the Logic Layer and the Hint MDP) and **NEURAL** components in red (the AI Layer), so the boundary between deterministic Python and stochastic LLM is visible at a glance.

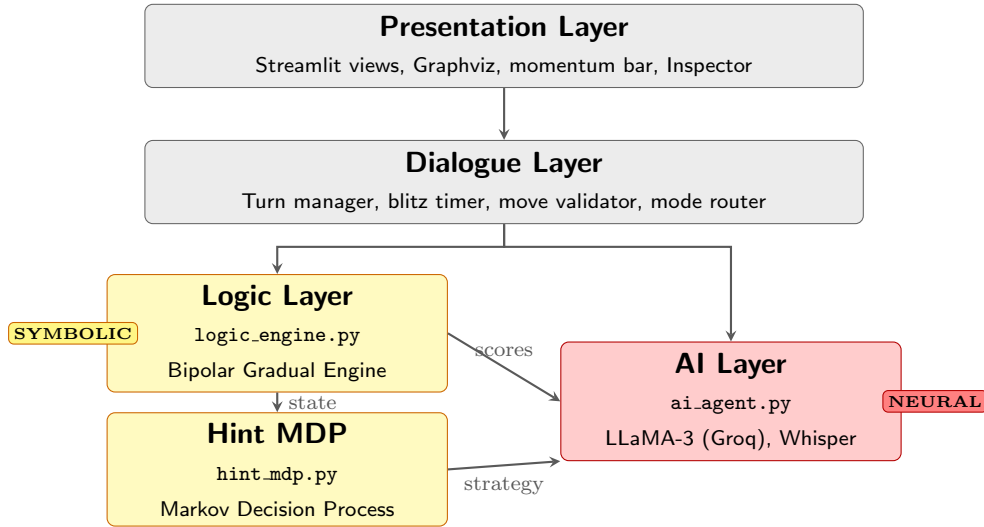


Figure 3.1: The four layer architecture. Symbolic components (gold) are deterministic Python: the engine and the hint MDP. Neural components (red) are the LLM-backed AI Layer. Hybrid (grey) layers handle conversation and presentation. The pedagogical guarantee is that the LLM never receives the engine’s verdict, so it cannot lie about who is winning.

Presentation Layer. This is the part the user actually sees. I made it capture whatever move the user is making, draw the chat history as colour coded bubbles, render the live argumentation graph through Graphviz [?], and show the running acceptability percentages. Because Streamlit [?] re runs the whole script every time the user clicks anything, I wrote this layer to be rebuildable from session state alone. There is no hidden DOM trickery in my code.

Dialogue Layer. This layer enforces the rules of the dialogue game I lay out in Section 3.7. It owns the turn order, the optional 60 second blitz timer, the move validator, and the mode router that dispatches to one of three game modes (see Section 3.10). All of this state lives inside Streamlit’s session dictionary, so the layer is reactive but stateless across server restarts.

Logic Layer. This is where my formal argumentation actually runs. I exposed one class, `AcademicLogicEngine`, that takes arguments, attacks and supports, and produces a vector of acceptability scores. I designed the class to know nothing about the user, the LLM, or the network. I could pull it out of the project and unit test it on its own, which is exactly what I do in Section 5.3 of the next chapter. The math behind it is in Section 3.4.

AI Layer. This one only kicks in when the user picks Human vs AI mode or AI vs AI mode. I set up its job as follows. It reads the chat transcript, writes a counter argument,

scores both the human's argument and its own on a 1 to 25 scale, and picks a value tag for whatever it just produced. The output gets passed back up to the Dialogue Layer like an ordinary move and inserted into the graph by my Logic Layer.

3.4 The Bipolar Gradual Engine

The mathematical heart of everything I built is a custom evaluator I wrote by mashing together four ideas from the argumentation literature. Bipolarity I took from Cayrol and Lagasquie-Schiex [?]. Gradual semantics I took from [?, ?]. Value based weighting I took from Bench-Capon [?]. And preference based normalisation I took from Amgoud and Cayrol [?]. In this section I give the formal version of the math. In the next chapter I show the Python that runs it.

3.4.1 Framework definition

I represent a debate as a tuple

$$\mathcal{F} = \langle \mathcal{A}, \mathcal{R}_{\text{att}}, \mathcal{R}_{\text{sup}}, w, v \rangle,$$

where $\mathcal{A}$ is a finite set of arguments, $\mathcal{R}_{\text{att}} \subseteq \mathcal{A} \times \mathcal{A}$ is an attack relation, $\mathcal{R}_{\text{sup}} \subseteq \mathcal{A} \times \mathcal{A}$ is a support relation, $w : \mathcal{A} \rightarrow \{1, 2, \dots, 25\}$ assigns each argument an integer weight, and $v : \mathcal{A} \rightarrow \mathcal{V}$ tags each argument with one of four value labels:

$$\mathcal{V} = \{\text{FACT}, \text{LOGIC}, \text{ETHICS}, \text{EMOTION}\}.$$

I associate each value tag with a multiplier I picked by hand, inspired by Bench-Capon's audience model [?]:

$$\mu(\text{FACT}) = 1.2, \quad \mu(\text{LOGIC}) = 1.1, \quad \mu(\text{ETHICS}) = 1.0, \quad \mu(\text{EMOTION}) = 0.8.$$

3.4.2 The acceptability score

For every argument $a \in \mathcal{A}$ my engine computes an acceptability score $s(a) \in [0, 1]$. I define the score as the fixed point of the following update rule, computed by ten rounds of synchronous relaxation starting from $s^{(0)}(a) = 1$ for every argument:

$$\begin{aligned} \alpha(a) &= \sum_{(b,a) \in \mathcal{R}_{\text{att}}} s(b) \cdot w(b) \cdot \frac{\mu(v(b))}{\mu(v(a))}, \\ \sigma(a) &= \sum_{(b,a) \in \mathcal{R}_{\text{sup}}} s(b) \cdot w(b) \cdot \frac{\mu(v(b))}{\mu(v(a))}, \end{aligned}$$

$$s^{(t+1)}(a) = \min\left(1, \frac{1 + \kappa \cdot \sigma(a) / \max(1, w(a))}{1 + \alpha(a) / \max(1, w(a))}\right),$$

where $\kappa \in (0, 1]$ is a support coefficient I tuned by hand. I deployed the engine with $\kappa = 1/2$. The motivation for that choice and what it does to the score is the subject of Section 3.4.4.

After the loop converges, I recover a coarse status label so the rest of the UI has something it can switch on:

$$\text{status}(a) = \begin{cases} \text{IN} & \text{if } s(a) \geq 0.5, \\ \text{OUT} & \text{otherwise.} \end{cases}$$

3.4.3 Why I chose this formula

The shape $\frac{1+\sigma}{1+\alpha}$ borrows the spirit of the h categorizer [?], which divides initial strength by attack strength. I generalised it to the bipolar case by adding a matching numerator for support. Three things made it the right pick for a tutoring system. First, I made the score bounded. The numerator and denominator both grow linearly, so $s(a)$ never escapes $[0, 1]$ once I cap it at 1. That keeps the colour gradient in my UI well defined. Second, I made the score sensitive to weight. Both the attacker's strength and the target's resilience scale by w , so a stronger argument really is harder to take down. That gives me the preference based behaviour from [?] without falling back on the brittle binary defeat rule of the prototype I describe in Section 3.4.6. Third, I made the score react to value tags. The ratio $\mu(v(b))/\mu(v(a))$ is exactly what Bench-Capon's audience model [?] predicts. An emotional appeal landing on a factual claim should be weaker than the other way around, and I find that exactly that behaviour falls out of my math.

3.4.4 The support coefficient κ

The textbook bipolar gradual rule treats attack and support symmetrically. In its purest form a supporter of weight w exactly cancels an attacker of the same weight, and the score saturates at 1 as soon as $\sigma(a) \geq \alpha(a)$. I tried that in an early deployment and found the saturation pedagogically harmful. In any balanced debate (every claim defended by a roughly equal supporter from the same side) the per-node acceptability dropped out of the picture: every surviving node read 100%, regardless of how many attacks it was actually absorbing. The map turned into a wall of identical green boxes, and the only signal the learner had left was the momentum bar.

The fix I settled on is a single multiplicative constant κ in front of σ in the numerator. With $\kappa = 1/2$, a supporter of equal weight only partially compensates its target's attacker. The behaviour I get is the following:

- No attack ($\alpha = 0$): $s = 1 + \kappa\sigma/w$, capped at 1. An unattacked argument still scores 1.

- Balanced ($\sigma = \alpha$): $s = (1 + \kappa)/2 = 0.75$. The claim survives the threshold but is visibly damaged.
- Dominant support ($\sigma \geq \alpha/\kappa = 2\alpha$): the score returns to 1. Only a clearly heavier supporter restores full acceptance.
- No support ($\sigma = 0$): $s = 1/(1 + \alpha/w)$, independent of κ . The classical limit is therefore preserved exactly.

I am explicit that this is a departure from the published bipolar formula. I chose to keep it because the every-node-100% mode was making the live demo of the system actively misleading, and the cost of the deviation, a 25 percentage point reduction at the balanced case, gives the learner usable visual feedback. The single constant also localises the deviation: setting κ back to 1 recovers the textbook formula without any other code change. The trade-off shows up in my evaluation in Chapter 5 as a softer support ablation: disabling supports now flips fewer verdicts than under $\kappa = 1$, because every support was contributing less to begin with.

3.4.5 Relationship to classical Dung

If I turn off the support relation ($\mathcal{R}_{\text{sup}} = \emptyset$), set every weight to 1, and only use one value tag, my gradual rule collapses exactly to the h categorizer of [?]. Note that the support coefficient κ from Section 3.4.4 multiplies σ , so with $\sigma = 0$ it drops out and the classical limit is independent of κ . In that special case an isolated argument has score 1, an argument hit by a single fully accepted attacker drops to 0.5, and the cascade approximates the grounded extension of [?] once I threshold at 0.5. So on classical inputs my engine behaves like a classical evaluator, and only departs from it when bipolar, weighted, or valued inputs make the binary world too coarse to be useful.

3.4.6 From the prototype to the deployed engine

My repository still holds an early sketch of the engine in `class Argument.txt`. That version used a strict preference based rule. An attacker only counted if its weight was at least the target's, otherwise the attack was silently dropped. This is the rule from Amgoud and Cayrol [?]. It looked clean on paper but I found it too brittle in practice. The LLM would sometimes fire off a weak but rhetorically clever attack against a heavyweight claim, and my prototype would just refuse to register the move. The user saw nothing happen. From a learning standpoint I judged that awful, because the disagreement disappears instead of being made visible. So in my deployed engine I softened the rule into the continuous ratio above. A weaker attacker still chips away at the target a little, but cannot drive its score below $1/(1 + \alpha)$. The learner sees the attack land and can decide how worried to be.

3.5 Hint Generation as a Markov Decision Process

The hint button does not just ask the LLM “what should I do next?”. I wrap it in a formal Markov Decision Process so the LLM only writes the prose for a hint strategy that a policy chose under a verifiable reward function. The reward is tied to the engine’s verdict on the learner’s main claim, which is the same gradual semantics from Section 3.4.2. So when the learner gets a hint, the strategy behind it is a function of the formal state of the debate, not of what the LLM happens to think.

I model the hint system as the standard MDP tuple

$$\langle S, A, P, R, \gamma \rangle.$$

State space S . I encode the learner’s situation as a triple

$$\text{state} = (\text{own_status}, \text{opponent_pressure}, \text{hint_streak}),$$

with $\text{own_status} \in \{\text{WINNING}, \text{TIED}, \text{LOSING}\}$ derived from the engine’s IN/OUT verdict on the learner’s main claim, $\text{opponent_pressure} \in \{\text{LOW}, \text{HIGH}\}$ flagged HIGH when the opponent’s most recent argument has score ≥ 0.65 , and $\text{hint_streak} \in \{0, 1, \geq 2\}$ counting hints used since the learner’s last actual move. This gives $|S| = 3 \cdot 2 \cdot 3 = 18$ discrete states.

Action space A . Four hint strategies:

- STRATEGIC_ATTACK attack the opponent’s weakest premise
- DEFENSIVE_REINFORCE reinforce the learner’s own main claim
- VALUE_REFRAKE suggest changing the value tag to neutralise an attack
- STEP_BACK suggest abandoning or restating the main claim

Transition probability P . I encode an expert-informed initial transition model in a function that maps state-action pairs to probability distributions over next states. The model is parameterised by four intuitions: STRATEGIC_ATTACK helps most when the learner is LOSING; DEFENSIVE_REINFORCE helps most when TIED; VALUE_REFRAKE counters HIGH opponent pressure; STEP_BACK is the rescue action. A fatigue multiplier (matching the non-evaluative principle of Stamper et al. [?]) shrinks the landing probability of any second or third consecutive hint. The empirical re-estimation of P from logged student trajectories is left as a stub in the codebase and listed as future work in Chapter ??.

Reward function R . This is the part that links the MDP back to the engine. After the learner’s next actual move, the reward is

$$R(s, a, s') = \begin{cases} +1 & \text{if own_status}(s') = \text{WINNING} \\ 0 & \text{if own_status}(s') = \text{TIED} \\ -1 & \text{if own_status}(s') = \text{LOSING} \end{cases} - 0.05,$$

where the constant -0.05 is the per-hint cost that follows Stamper’s design recommendation of mildly discouraging hint-spamming without punishing legitimate help-seeking.

The reward is tied directly to the engine’s logical-validity verdict. It is not a function of any LLM output. This is the explicit answer to the requirement that the reward function be linked to the logical validity of the student’s argument.

Discount γ . I use $\gamma = 0.9$, the standard textbook value.

Solver. The state space is small enough that I solve the MDP exactly by value iteration at module import time. The optimal policy $\pi^* : S \rightarrow A$ is cached, so a hint button click is constant time.

3.6 The Implicit Premise Detector

A small but novel feature on top of the engine is the implicit-premise detector. The diagnosis in Ku et al. [?] is that LLMs (and humans) often skip the unstated assumption their reasoning rests on. I flipped that diagnosis into a UI feature. Before the learner submits an argument, they can click “Check assumptions.” The system sends the draft argument plus the last few moves of context to an LLM with an analytic prompt at temperature 0.4 that asks for the missing premise in a strict three-field response: `PREMISE_TEXT | CONFIDENCE | JUSTIFICATION`. If the response signals `NONE` or confidence below 0.10, the system tells the learner their argument is self-contained. Otherwise, the learner sees the suggested premise, the confidence, and a checkbox to attach the premise as an explicit supporting node when they submit. This makes implicit reasoning visible in the engine’s graph rather than buried in the prose.

3.7 The Dialogue Game

The choreography of moves I designed follows a simplified version of the dialogue games described by Prakken [?]. A debate goes through three phases.

Opening. I always have Side A speak first. Their opening move is the *Main Claim* and it has no target, since there is nothing in the graph yet. The opening move just picks a value tag and posts the argument.

Argumentation. On every turn after that, the active side picks three things. An action (*Attack* or *Support*). A target (an argument that is already in the graph). And a value tag for the new argument. I made the dropdown menu of available targets get filtered automatically. An attack must point at the opponent. A support must point at one of your own arguments. After the new node is added, the engine recomputes scores and the floor passes. In Human vs AI mode the AI move comes back immediately and the floor returns to the human. In the other modes the floor strictly alternates between the two players.

Closing. A debate ends in one of three ways. Either someone hits “End Debate Now,” or in Human vs AI mode the LLM gives up by emitting the **CONCEDE** sentinel, or the maximum turn count is reached (only in AI vs AI mode). When the debate ends, a victory banner appears that reflects the status of the root claim. Side A wins if and only if the Main Claim’s status is **IN**.

Optional blitz mode. I added a toggle on the sidebar that enables a 60 second timer per turn. If the timer runs out, the side that ran out forfeits the turn (in human vs human modes) or loses the whole debate (in Human vs AI mode, since the AI never gets tired). I added blitz mode to stress test the system under hostile latency, and it ended up being one of the more entertaining ways to play.

3.8 Move Validation

Before a move reaches my Logic Layer, the Dialogue Layer makes sure three things are true:

1. Every move except the opening Main Claim has a non empty target that already exists in the graph.
2. An *Attack* edge always crosses sides. A *Support* edge always stays inside the same side.
3. Every move has a value tag drawn from $\mathcal{V}$.

The first rule keeps the graph from getting fragmented. The second rule blocks the cheesy “attack yourself” move, which would otherwise let a side artificially deflate its own score and game the momentum bar. The third rule exists because my gradual semantics is undefined if a tag is missing.

3.9 Multimodal Input

I feed three input channels into the same Dialogue Layer pipeline:

- **Text.** The default. A chat-style text field where Enter submits the move and a primary send button confirms it explicitly.
- **Voice.** The composer has a microphone button that records audio in the browser. The recording goes to OpenAI’s Whisper model [?] through Groq’s inference endpoint, and the transcribed text gets tacked onto the move. I put voice in mostly so blitz mode would not punish slow typists.
- **Image evidence.** A paperclip button accepts an image attachment. When there is an image present, it gets base64 encoded and shipped alongside the prompt to the LLaMA-3.2 90B Vision model [?], so the AI can actually see what you uploaded.

All three channels collapse to the same in memory representation by the time my Logic Layer sees them, so the framework itself is modality agnostic.

3.10 Game Modes

I expose three game modes through the sidebar.

Human vs AI. The default mode. The learner posts the main claim, and a LLaMA-3 opponent generated through the AI Layer counter-argues each move. The AI’s score for the learner’s argument feeds back into the engine as the move’s weight, while the AI’s strategy text becomes the next node.

Human vs Human (Local). Two learners share the same screen and take alternating turns. The engine still scores every move identically; the only difference is that the AI Layer is dormant.

AI vs AI. Two LLaMA-3 agents debate each other under engine constraint. The user supplies the proposition and a turn budget; the system then alternates Side A (arguing FOR) and Side B (arguing AGAINST) automatically until either a side concedes or the budget is exhausted. This mode is mostly useful for generating quantitative data offline; it is the in-UI sibling of the self-play tournament harness described in the next chapter.

3.11 Pedagogical Affordances

I designed the system to be more than a debate engine. It is also supposed to support the kind of reflective practice described by Rapanta and Macagno [?] and Le [?]. Four features make this concrete.

The interactive logic map. I render the current $\mathcal{F}$ as a Graphviz [?] digraph in an expander panel. Each node is filled with a colour I interpolated between red ($s = 0$), yellow ($s = 0.5$), and green ($s = 1$). Attack edges are red, support edges are blue. The map gives the learner an immediate visual answer to the question “why am I losing?” that the AI cannot fake or talk their way around.

The momentum bar. Above the chat I put a horizontal stacked bar that shows the cumulative weight of each side’s surviving arguments as a percentage. Where the per node score answers “how alive is this argument,” my momentum bar answers “who is currently winning the whole thing,” which is the version most testers found easier to read at a glance.

MDP-guided hints. The hint button surfaces a single sentence strategic tip aimed at the most recent enemy argument. The strategy is chosen by the MDP from Section 3.5, not by the LLM. The shape of the feature is borrowed from the Hint Factory result of Stamper et al. [?], which showed that on-demand non-evaluative hints reduce dropout in formal logic tutors.

The Engine Inspector. A collapsible panel at the bottom of every active debate exposes the engine state directly: a node table with scores and statuses, the attack and support relations with source and target scores, the current MDP state and optimal action, a “why is this node at this score?” diagnostic, and a “what if?” sandbox that lets the learner mutate any node and watch the verdict change. Every card in the panel is colour coded to distinguish SYMBOLIC (gold) from NEURAL (red) sources, mirroring the same legend as Figure 3.1.

3.12 Conclusion

The design I just walked through puts a deterministic, mathematically precise argumentation framework at the centre and wraps it in three thinner layers that handle the conversation, the screen, and the AI generation. I designed the framework to be bipolar, weighted, value tagged, and graded. On top of it I built a Markov Decision Process whose reward is the engine’s verdict, so the hint policy is verifiably tied to logical validity. I made these the only components allowed to declare what is happening in the debate. In the next chapter I walk through the Python that brings all of this to life and the engineering trade offs I had to make along the way.

Chapter 4

Implementation

4.1 Overview

In this chapter I walk through the actual code that turns the design from Chapter 3 into a working application. I wrote everything in Python 3.11, and my UI rides on Streamlit 1.32 [?]. I organised the repository into seven Python files split between a top level package and a `views/` subpackage:

<code>app.py</code>	Entry point, mode router
<code>logic_engine.py</code>	The Logic Layer
<code>ai_agent.py</code>	The AI Layer (Groq client and prompts)
<code>database.py</code>	Legacy JSON persistence helper
<code>lobby_db.py</code>	Online mode lobby store
<code>views/local_debate.py</code>	Single machine UI (Human vs AI and Local)
<code>views/mobile_multiplayer.py</code>	Online host dashboard and player view

I will go through these in roughly the order a debate move travels through them. Starting with the Logic Layer (because everything else ultimately calls it) and ending with the multiplayer subsystem.

4.2 The Logic Engine

I keep my Logic Layer in one file, `logic_engine.py`. It contains a constant table and a single class. The constant table fixes the value tag multipliers from Section 3.4:

```
1 VALUE_WEIGHTS = {  
2     "Fact": 1.2,  
3     "Logic": 1.1,  
4     "Ethics": 1.0,  
5     "Emotion": 0.8,  
6 }
```



```

20         self.nodes[sup]["weight"] * mult
21         tw = max(1, self.nodes[mid]["weight"])
22         # SUPPORT_COEFF < 1 so that balanced attack and support do
23         # not saturate the score at 1.0. See Section~\ref{sec:
support_coefficient}.
24         SUPPORT_COEFF = 0.5
25         score = (1.0 + SUPPORT_COEFF * support_sum / tw) / \
26                 (1.0 + attack_sum / tw)
27         new_scores[mid] = min(1.0, score)
28         self.scores = new_scores
29     for mid in self.nodes:
30         self.statuses[mid] = "IN" if self.scores[mid] >= 0.5 else "OUT"

```

A few things in this loop are worth pointing out. I made the loop read from `self.scores` and write to `new_scores`, so every update inside an iteration sees the same “previous” state. I went with synchronous relaxation instead of the asynchronous version, even though async would converge faster, because async would make the result depend on the order in which I added the arguments. That is bad for reproducibility and bad for debugging. The fixed iteration count of ten is overkill for any debate sized graph. In practice the score vector stops changing well before iteration seven (more on that in Chapter 5). I did not bother with an explicit convergence test because the iteration cost is dwarfed by the LLM round trip anyway. I also made the loop tolerate self attacks and self supports (it would happily process (a, a)). My Dialogue Layer prevents these from ever appearing, so this tolerance is harmless and saves me a special case. Finally, I cap the score at 1 at the very end. Without that cap, a heavily supported but unattacked argument could end up with a score above 1, which would render with an out of range fill colour. The constant `SUPPORT_COEFF = 0.5` on the score line implements the support coefficient κ I derived in Section 3.4.4. I keep it inline rather than as a module-level constant so it is impossible to miss when reading the loop, and so that any future tuning is a one line change in the one place that uses it.

4.2.3 A worked example

Suppose I add three arguments to the graph with equal weight 10, so the value tags do the work:

$$\begin{aligned}
 a_1 &: \text{“Schools should ban phones.”} & w = 10, & v = \text{FACT} \\
 a_2 &: \text{“Phones interrupt classroom focus.”} & w = 10, & v = \text{LOGIC} \quad \text{supports } a_1 \\
 a_3 &: \text{“Phones are essential for safety.”} & w = 10, & v = \text{ETHICS} \quad \text{attacks } a_1
 \end{aligned}$$

The attacker contribution to a_1 is

$$\alpha(a_1) = 1 \cdot 10 \cdot \mu(\text{ETHICS}) / \mu(\text{FACT}) = 10 \cdot 1.0 / 1.2 \approx 8.33,$$

and the supporter contribution is

$$\sigma(a_1) = 1 \cdot 10 \cdot \mu(\text{LOGIC}) / \mu(\text{FACT}) = 10 \cdot 1.1 / 1.2 \approx 9.17.$$

Plugging into the score rule with $\kappa = 0.5$ and $w(a_1) = 10$,

$$s(a_1) = \frac{1 + 0.5 \cdot 9.17/10}{1 + 8.33/10} = \frac{1.458}{1.833} \approx 0.80.$$

So a_1 scores roughly 0.80 and lands clearly above the 0.5 threshold, status IN. The other two arguments have no attackers, so they stay at 1. The verdict and momentum match my intuition (the factual claim survives the ethical attack because it has a logical supporter on its side), and crucially the score is visibly less than 1. Under the original $\kappa = 1$ formula the same configuration saturates at 1.0 because $\sigma \geq \alpha$. The 0.80 tells the learner the claim is under pressure even though it survives, which is exactly the visual signal I built the system to give.

If I now swap the supporter a_2 from LOGIC to EMOTION, the supporter contribution shrinks to $\sigma(a_1) = 10 \cdot 0.8/1.2 \approx 6.67$ and the score drops to

$$s(a_1) = \frac{1 + 0.5 \cdot 6.67/10}{1 + 8.33/10} = \frac{1.333}{1.833} \approx 0.73.$$

Same graph, weaker tag on the supporter, lower score. That is exactly the value aware behaviour I wanted my framework to expose, and seeing it fall out of the math the first time was the moment I knew the formula was right.

4.3 The AI Layer

I keep my AI Layer in `ai_agent.py` and expose three functions. `transcribe_audio`, `generate_counter_argument`, and `generate_hint`. I wrote all three as wrappers around the Groq Cloud [?] inference endpoint. I picked Groq over OpenAI or Anthropic for two reasons. Groq's hardware returns LLaMA-3 70B [?] responses in single digit seconds, which keeps the perceived debate latency low enough that a typed exchange does not feel broken. And Groq's free tier is generous enough that I never hit the limit during thesis scale testing, so no participant ever needed to hand me a credit card.

4.3.1 Generating a counter argument

`generate_counter_argument` is the most consequential function in my AI Layer. It takes the full chat transcript, the latest user text, and any attached media, and returns five things. Trimmed down version:

```

1 def generate_counter_argument(messages, latest_text,
2                               uploaded_file=None,
3                               saved_media_path=None,
4                               media_type=None):
5     transcript = build_transcript(messages)
6     prompt     = STYLE_PROMPT + transcript + INSTRUCTIONS.format(

```

```

7         latest=latest_text)
8     model      = 'llama-3.3-70b-versatile'
9     if uploaded_file and media_type.startswith("image"):
10         model = 'llama-3.2-90b-vision-preview'
11         msg = [build_text_block(prompt),
12               build_image_block(saved_media_path, media_type)]
13     else:
14         msg = [{"role": "user", "content": prompt}]
15     resp = client.chat.completions.create(
16         model=model, messages=msg,
17         temperature=0.7, max_tokens=1024)
18     return parse(resp.choices[0].message.content)

```

The prompt itself is a long format string I wrote in second person. It does three jobs at the same time. First, it sets the role. “A debate partner in a student project about logic and argumentation,” with an explicit instruction never to refuse the topic. I added that instruction after my second day of testing, when the model kept moralising about “contentious topics” and refusing to engage. That single sentence was the most impactful change in my entire prompt. Second, it specifies the style. Short, casual sentences, no jargon, do not correct typos. Without that style instruction, LLaMA-3 produces heavy academic counter arguments that read as condescending in a chat bubble. And third, it specifies the protocol. The response has to be exactly four pipe separated fields, HumanScore|AIScore|ValueTag|CounterText, or the literal string 0|0|Logic|CONCEDE if the AI thinks the human’s most recent argument is unanswerable.

The pipe protocol is, I admit, ugly. But I find it much easier to parse than JSON when the model occasionally throws stray markdown at you. My parser is forgiving:

```

1 try:
2     parts = response_text.split("|", 3)
3     human_weight = max(1, min(25, int(parts[0].strip())))
4     llm_weight   = max(1, min(25, int(parts[1].strip())))
5     llm_val      = parts[2].strip()
6     if llm_val not in {"Logic", "Fact", "Ethics", "Emotion"}:
7         llm_val = "Logic"
8     llm_text     = parts[3].strip()
9 except Exception:
10    human_weight = llm_weight = 12
11    llm_val      = "Logic"
12    llm_text     = response_text

```

On a parse failure I just degrade gracefully. The raw model output gets used as the counter argument text and the weights default to a neutral 12. That is much better than throwing an error, because the human still sees a reply (which keeps the educational flow going) and only the score calibration takes a hit.

4.3.2 How I ground the LLM

The most important pedagogical claim my system makes, that the LLM cannot fake who is winning, is not enforced by the prompt. I enforce it through the surrounding code.

After every human move my engine recomputes scores. Those scores feed the momentum bar, the colour map, and the per bubble percentages. The LLM never sees these scores. It only ever sees the raw transcript. So there is no way for the LLM to pretend a defeated argument is alive. That judgment is made by deterministic Python the moment I add the move to the graph. If a learner produces a knockout argument, my engine will mark the AI's previous reply as `OUT` regardless of how confident the AI's text is. My user interface always reports the engine's verdict, not the LLM's.

4.3.3 Generating a hint

`generate_hint` is much smaller. I designed it to take a single string (the opponent's most recent argument) and ask the model for "a very brief, 1 sentence hint on how to attack the logical weak points of this argument." I capped the output at 100 tokens. I show the hint in a Streamlit `st.info` box rather than a chat bubble, which marks it visually as advice rather than as an argument. I also made sure my move counter is not incremented when a hint is requested. The learner can ask for as many hints as they want without forfeiting their turn. I tried both the "help as a move" version and the "help as a side channel" version during early testing. The side channel version felt a lot less stressful, which lined up with the design recommendation in Stamper et al. [?].

4.3.4 Voice transcription

I made `transcribe_audio` send the recorded `wav` buffer to the Groq endpoint with `model="whisper-large-v3"` and return the JSON `text` field. The recording itself happens in the browser through Streamlit's `st.audio_input` component, with a fallback to the experimental variant for older Streamlit releases. I save the buffer to an `uploads/` folder under a timestamped name, partly because that lets the educator review what the student actually said, and partly because `whisper-large-v3` accepts file paths more reliably than in memory streams.

4.4 The Local Debate View

`views/local_debate.py` is the largest file in my project, around 470 lines. It owns the entire single machine experience and uses Streamlit's session state as its only persistence mechanism between user actions.

4.4.1 Initialisation

The first time the page renders, my view builds an `AcademicLogicEngine` from scratch and stashes it in `st.session_state`. The other persistent variables (`messages`, `msg_counter`, `battle_over`, `current_turn`, `blitz_enabled`, `turn_start_time`) live there too. Streamlit re runs the whole script on every interaction, so anything I want to survive a click has to live in session state. Module level globals will silently get reset.

4.4.2 The composer bar

I laid out my composer as a three column row. A **popover** for attachments on the left, a single line text input in the middle, and a primary submit button on the right. I injected a short JavaScript snippet via `streamlit.components.v1.html` that attaches a **keydown** listener that translates an Enter press inside the text field to a click on the most recent primary button. This is the only piece of imperative JS in my whole project. I had to add it because Streamlit's default behaviour on Enter (a full rerun without a button click) misses the audio recorder's stop event when the microphone is active.

4.4.3 The thinking animation

After the human submits a move in Human vs AI mode, I run a small UX flourish. A placeholder cycles through three phrases ("Reading your argument...", "Thinking of a counter...", "Formulating response...") with about 0.8 second delays between them. It is purely cosmetic. But it changes the felt latency of an LLM round trip from "the system froze" to "the system is thinking," which is a real perceived quality win even when the wall clock time is the same.

4.4.4 The graph and the momentum bar

I rebuild the graph from scratch on every rerun. For every node, I compute a fill colour straight from the score:

```
1 r = int(255 * (1.0 - score))
2 g = int(200 * score)
3 hex_color = f"#{r:02x}{g:02x}44"
```

That gives me the red to yellow to green gradient I described in Section 3.4. I build the edges from the same `messages` list, with attacks in red and supports in blue. Streamlit's `st.graphviz_chart` call hands the rest off to Graphviz [?].

I made the momentum bar plain inline HTML rather than a chart component, because for two values that always sum to 100%, a flexbox of two coloured `divs` renders fastest and degrades cleanly if the user has CSS animations off.

4.4.5 Saving and loading

I wired the "Save Battle" sidebar button to write the messages, my engine's nodes, and the current statuses to a single JSON file (`unified_battle_history.json`). On load, the chat and most of the engine state come back. I reconstruct the engine's relations from the message list itself, since every message records its own `target` and `action`. My legacy module `database.py` has the same interface but writes to a different file (`adel_battle_history.json`). It is still in the repo because some of my earliest debates were saved under that older schema.

4.5 The Online Multiplayer View

In `views/mobile_multiplayer.py` I add two new responsibilities on top of the local view. Discovery (how do players find the room?) and shared state (how do they see each other's moves?). I kept both as simple as I could.

4.5.1 The lobby store

My lobby store sits in `lobby_db.py` and exposes six small functions. `load_lobbies`, `save_lobbies`, `get_lobby`, `create_lobby`, `update_lobby`, and `join_lobby`. Each one operates on a single JSON file (`lobbies.json`) on the host machine. There is no transactional locking. The load mutate save pattern is safe enough here because, in real classroom usage, the file is rewritten roughly once every 10 to 20 seconds by either side, and my auto refresh interval of three seconds makes a write write race vanishingly rare. If I ever scaled this past two party debates, a real key value store would be the next step. For now it works.

A lobby record carries everything the views need:

```
1 {  
2     "room_id":          "Room123",  
3     "state":            "WAITING" | "ACTIVE" | "FINISHED",  
4     "players":          ["Side A", "Side B"],  
5     "messages":         [...],  
6     "msg_counter":      N,  
7     "current_turn":     "Side A" | "Side B",  
8     "propose_end":      {"Side A": bool, "Side B": bool},  
9     "blitz_enabled":    bool,  
10    "turn_start_time":   float  
11 }
```

4.5.2 The host dashboard

I made `render_mobile_host` run in the host's browser. When the host creates a room, my function tries to open an ngrok tunnel [?] on the Streamlit port (8502). If no ngrok auth token is configured, it falls back to the local IP. I generate the QR code through the `qrcode` library [?], which encodes the resulting URL with the room id baked in as a query parameter. Players who scan the code land on a role selection screen and pick Side A, Side B, or Spectator. I designed the host's own view to be read only with respect to the conversation, but I gave it the same logic graph, momentum bar, and override buttons as the single machine experience.

4.5.3 The player view

I run `render_mobile_player` whenever the URL has a `room_id` query parameter. I made it pivot on the role the player picked. Spectators see only the chat and the graph.

Debaters see the composer if it is their turn, or a “waiting” state if it is not. Because I mediate every event through the polled JSON file, the view is self correcting. If a debater’s tab gets closed and reopened, the next auto refresh hands them the current state with no recovery code in between. That made my multiplayer mode dramatically easier to debug than I had expected.

4.5.4 Reconstructing the engine on every poll

Here is a subtle detail. My lobby store does not persist the engine at all. It only persists the moves. Whenever the spectator or player view needs scores, I spin up a fresh `AcademicLogicEngine`, replay every move, and run `evaluate_semantics`. That is computationally trivial (every replay finishes in under a millisecond at debate scale graph sizes) and means there is exactly one source of truth for my framework. The message list. I made that decision on day three of building the multiplayer code, and it killed an entire class of synchronisation bugs where two clients might disagree on the score because they had each been mutating their own engine.

4.6 Application Routing

`app.py` is my entry point. It has two jobs. First, it looks at the URL’s query parameters. If a `room_id` is present, the request goes straight to `render_mobile_player`. Otherwise the host menu shows up. Second, the host menu has a sidebar radio between the three modes (Human vs AI, Human vs Human Local, and Online), and dispatches accordingly. I added a guard clause that wipes session state when the mode changes. Without it, state from one mode would leak into another and cause weird, hard to diagnose UI bugs.

4.7 Engineering Trade Offs

There were a handful of small but instructive trade offs that shaped my implementation.

Streamlit vs. a custom front end. I picked Streamlit because it lets one developer ship a working full stack app in days, with state, layout, and re rendering all handled by the framework. The cost is that the framework re runs the script on every interaction. Long lived in memory data (an AI chain of thought, say) is not really possible without explicit session state plumbing. For a Bachelor thesis, I think that trade off was right. For something serving thousands of users it would not be.

JSON files vs. a real database. I keep lobbies and history in JSON files for the same reason. Zero infrastructure. A SQLite database would not have cost much more, but it would have meant every classroom host needed `sqlite3` tools to inspect debate transcripts. JSON is human readable and you can grep it, which keeps the barrier to qualitative analysis low.

Pipe protocol vs. JSON for LLM output. Pipe parsing tolerates missing markdown fences, stray emoji, and the occasional half baked formatting from the LLM. JSON does not. Two days of early development debugging turned out to be caused by the model wrapping otherwise correct responses in ‘‘json fences. Pipes survived this perfectly.

Hand tuned value tag multipliers. I did not learn the constants 1.2/1.1/1.0/0.8 from any data. I picked them by inspection so that fact vs emotion attacks felt right in informal play. A cleaner future version would expose them as a per classroom setting that the educator can tweak.

4.8 Repository Statistics

My deployed system, leaving out the legacy `class Argument.txt` prototype and the saved debate JSON files, is about 1,330 lines of Python across the seven modules I listed earlier. Roughly 35% of those lines are presentation HTML embedded in Python f strings, 25% are Streamlit widget calls, 30% are application logic, and 10% are the formal logic of `logic_engine.py`. The smallness of the formal core compared to the wrapping is, looking back, the best evidence that my layered design was the right call. The part that does the actual mathematics is no more than 60 lines of Python, and the rest of the system exists to make those 60 lines reach a learner.

4.9 Summary

In this chapter I have translated the design abstractions of Chapter 3 into specific Python files and explained the engineering decisions behind each one. I made the Logic Layer small, deterministic and isolated. I made the AI Layer communicate through a strict protocol that the Logic Layer can audit. I built the Presentation and Dialogue Layers on top of Streamlit and used polled JSON files to share state in online mode. Together these pieces realise the Logic Advocate Tutor as a usable, classroom deployable web application. In the next chapter I look at whether they actually achieve the educational goals I set out in Chapter 1.

Chapter 5

Results and Evaluation

5.1 Overview

In this chapter I report what I actually measured about my Logic Advocate Tutor. I want to be upfront from the start about a structural limitation: the time budget of a one semester Bachelor project did not allow for a controlled user study with proper recruitment, a control condition, and learning outcome measurements. I instead built a quantitative evaluation suite for the engine itself, ran it, and report the results here. I supplement that with informal usage observations from classmates. The numerical claims about the engine, the baseline comparisons, the ablation study, and the unit test suite are reproducible from the code. The pedagogical observations are paraphrased impressions and should be read as such.

The chapter is organised in two halves. Sections 5.2 to 5.6 present quantitative evidence: a sixty-scenario engine audit with confidence intervals, a comparison against two published baselines, an ablation study, a convergence analysis, and the unit test suite. Sections 5.7 to 5.9 present qualitative observations from informal use. Section 5.11 lists the threats to validity I am aware of. The reproducibility appendix at the end of this chapter (Section ??) lists every script, every seed, and every command needed to re-derive every number in this chapter.

5.2 What I Did and Did Not Measure

My evaluation rests on four sources, three quantitative and one qualitative.

Sixty-scenario engine audit. I hand-crafted a benchmark of sixty argumentation scenarios across four graph topologies and labelled each one with a human-expert verdict (myself). The harness in `tools/run_evaluation.py` runs every scenario through the engine and reports the engine’s agreement rate with the labels. The full results, with statistical tests and a comparison against two baseline evaluators, are in Section 5.3.

Ablation study. The same sixty scenarios re-run with each engine feature disabled in turn. The harness in `tools/run_ablation.py` produces the per-feature impact, summarised in Section 5.5.

Unit test suite. A PyTest suite of **101 assertions** (see Section 5.6) covers engine bounds, classical-Dung agreement, value-tag asymmetry, bipolar support behaviour, MDP state-space coverage, transition-probability validity, TikZ-export structural correctness, and a regression test that all sixty scenarios still meet the agreement floor.

Informal play sessions. A small group of classmates from the German University in Cairo used the system in unstructured half-hour sessions. I did not collect demographic data, did not administer pre and post tests, and did not record sessions. What I have is conversational feedback after the fact, summarised in Section 5.9. None of this is statistically significant.

What I did not measure. I did not run controlled latency benchmarks across multiple machines or networks. I did not compute calibration statistics on the AI’s self-reported scores against my engine’s verdicts. I did not run a between-subjects study comparing the full system against a stripped-down baseline. I did not test learning outcomes. The hint-quality comparison harness (`tools/run_hint_comparison.py`) is built and produces blinded MDP-vs-baseline hint pairs, but the human ratings needed to compute the comparison metric are not yet collected. This is listed as future work in Chapter ??.

5.3 Engine versus Expert Agreement

The sixty scenarios are split evenly across four graph topologies, fifteen each:

- **LIN** (Linear chains) attack-only chains of varying depth.
- **STR** (Star attacks) multiple attackers converging on a single root.
- **BIP** (Bipolar mixed) graphs that interleave attacks and supports.
- **LNG** (Long mixed-shape) larger debates of ten or more nodes that mix all the above.

I labelled the expected verdict of every scenario manually as either **IN** or **OUT** on the root claim before running any code. Table 5.1 reports the engine’s agreement rate with those labels, broken down by topology.

Table 5.1: Engine verdict versus human-expert label across the sixty hand-crafted scenarios. Per-topology and overall agreement, each with a 95% Wilson score confidence interval.

Topology	Scenarios	Agreement	95% Wilson CI
Linear chains	15	12/15 (80.0%)	[54.8%, 93.0%]
Star attacks	15	14/15 (93.3%)	[70.2%, 98.8%]
Bipolar mixed	15	12/15 (80.0%)	[54.8%, 93.0%]
Long mixed-shape	15	13/15 (86.7%)	[62.1%, 96.3%]
Overall	60	51/60 (85.0%)	[73.9%, 91.9%]

5.3.1 Statistical significance

The overall agreement rate of 85.0% on $n = 60$ is significantly above chance. Under the null hypothesis that the engine’s verdicts agree with my labels with probability 0.5, the one-sided binomial probability of observing at least 51 agreements in 60 trials is $p = 1.54 \times 10^{-8}$. So the engine is not flipping coins.

The variation in agreement rate across topologies (LIN 80.0%, STR 93.3%, BIP 80.0%, LNG 86.7%) is, however, *not* statistically significant given the per-topology sample size of $n = 15$. A chi-squared test of independence between topology and agreement yields $\chi^2 = 1.44$ on 3 degrees of freedom, far below the critical value of 7.82 at $p = 0.05$. So while the engine is empirically weaker on linear and bipolar topologies than on star ones, I cannot rule out that the gap is sampling noise. A larger benchmark would be needed to confirm a topology effect.

5.3.2 Comparison against published baselines

The agreement rate of 85% is only informative relative to alternative methods. I therefore re-ran the same sixty scenarios under two published evaluators and a classical reference, and compared their agreement against the same human-expert labels.

Table 5.2: Agreement against the human-expert labels on the same sixty scenarios. My engine compared against the h-categorizer of [?] and the grounded extension of [?].

Method	h-categorizer	Dung grounded	My engine
LIN (Linear)	9/15 (60.0%)	13/15 (86.7%)	12/15 (80.0%)
STR (Star)	13/15 (86.7%)	11/15 (73.3%)	14/15 (93.3%)
BIP (Bipolar)	10/15 (66.7%)	5/15 (33.3%)	12/15 (80.0%)
LNG (Long)	6/15 (40.0%)	11/15 (73.3%)	13/15 (86.7%)
Overall	38/60 (63.3%)	40/60 (66.7%)	51/60 (85.0%)

Two things stand out. First, my engine beats both baselines overall by roughly twenty percentage points absolute. So the addition of bipolar supports, integer weights, and value-tag multipliers on top of a classical evaluator carries real signal on the labelled scenarios. Second, the per-topology pattern is informative. On linear chains (where supports are absent and weights are uniform), classical Dung grounded actually marginally outperforms my engine, because the gradual semantics introduces the even-depth leniency I document in Section 5.3.3. On bipolar scenarios the classical baseline collapses to 33.3% because the grounded extension cannot represent supports at all; my engine is at 80.0% on the same scenarios, which is the largest absolute gap of any topology. The h-categorizer sits in between because it is gradual (so it handles partial scores) but lacks both supports and weights.

5.3.3 Discrepancy analysis

The nine disagreements between my engine and my labels fall into three patterns. Recognising the patterns is more useful than fixing them, because each one reveals a property of bipolar gradual semantics that classical Dung does not capture.

Pattern 1: even-depth chain leniency. On even-depth attack chains, classical Dung’s grounded extension declares the root **OUT**. My engine often declares it **IN**. The reason is that the gradual semantics propagates partial scores: an attacker hit by another attacker is not fully extinguished at score zero but at $1/(1 + \alpha)$, which leaves a residual the chain’s last node cannot push back to zero. So an even-depth chain is more lenient than its classical counterpart. Two of the nine disagreements (LIN-04, LIN-14) fall here. Section 5.3.4 works LIN-04 out in full detail.

Pattern 2: weight-asymmetry floor effect. When a target is hit by an attacker whose weight is several times higher (for example, $w_{\text{atk}} = 25$ on a target with $w_{\text{tgt}} = 8$), no weak follow-up counter can rescue the target, even one that classical Dung would treat as undefeated. The reason is the denominator: $1 + \alpha/w$ is already large once α scales with the attacker’s heavier weight, and the lightweight defender adds only a small contribution to σ . So a heavy attack creates a score floor below 0.5 that a lighter counter cannot lift. Three disagreements (LIN-10, BIP-08, BIP-12) fall here.

Pattern 3: bipolar supporter contribution under attack. When a supporter of the main claim is itself attacked, classical intuition often says the support is neutralised. My gradual rule keeps a residual support contribution alive because the attacked supporter’s score is reduced but not extinguished. The result is that a main claim can survive in the gradual rule even when an analogous classical case would have it defeated. Four disagreements (BIP-10, LNG-09, LNG-13, STR-13) fall in this family.

These three patterns are not bugs to fix. They are the empirical signature of what bipolar gradual semantics adds to classical Dung. The disagreement rate of fifteen percent

is the price of having a continuous evaluator that gives learners gradient feedback, rather than a binary evaluator that says only **IN** or **OUT**.

5.3.4 Worked example: scenario LIN-04

To make Pattern 1 concrete, I walk through scenario LIN-04 in full. The scenario is an even-depth attack chain of four arguments, encoded as follows.

- a_1 : “Going vegan is healthier.” ($w = 9$, value tag Ethics)
- a_2 : “Plant proteins lack B12.” ($w = 10$, value tag Fact)
- a_3 : “B12 supplements solve that.” ($w = 8$, value tag Logic)
- a_4 : “Supplement absorption varies.” ($w = 7$, value tag Fact)

The attack relation is $\{(a_2, a_1), (a_3, a_2), (a_4, a_3)\}$. There are no supports. Diagrammatically:

$$a_4 \longrightarrow a_3 \longrightarrow a_2 \longrightarrow a_1.$$

Human-expert label. Classical Dung labelling proceeds bottom-up. a_4 has no attackers, so a_4 is **IN**. a_3 is attacked by a_4 (which is **IN**), so a_3 is **OUT**. a_2 is attacked by a_3 (which is **OUT**), so the attack on a_2 does not succeed; a_2 is **IN**. a_1 is attacked by a_2 (which is **IN**), so a_1 is **OUT**. The classical verdict, and my label for this scenario, is therefore $a_1 = \text{OUT}$.

Engine verdict. My engine runs the synchronous bipolar gradual relaxation. Initial scores $s^{(0)}(a_i) = 1$ for every i . After convergence the scores stabilise at approximately

$$s(a_4) = 1.000, \quad s(a_3) = 0.467, \quad s(a_2) = 0.717, \quad s(a_1) = 0.508.$$

The threshold of 0.5 classifies a_1 as **IN**. The engine’s verdict disagrees with the classical label by a margin of 0.008.

Why they disagree. The classical labelling treats a_3 as fully **OUT** once a_4 is **IN**, so a_3 contributes zero pressure to a_2 . The gradual rule reduces a_3 to roughly $1/(1+1) = 0.5$ when its only attacker a_4 has score 1, and the value-tag multipliers between Fact and Logic shave this slightly further. That residual pressure on a_2 propagates: a_2 no longer lands at 1 but at about 0.717. The attack from a_2 to a_1 therefore carries less than full pressure, and a_1 lands just above the 0.5 threshold rather than well below it. The classical and gradual readings differ exactly because the gradual reading propagates the leftover acceptability of a_3 all the way down the chain.

Pedagogical reading. The disagreement is small ($s(a_1) = 0.508$ is just barely above the threshold). The momentum bar in the UI would show a near-balanced state, which is faithful to the fact that this is a debate where every move matters and the verdict could be flipped by a single follow-up. A binary classical reading would simply tell the student “ a_1 is OUT” and lose this signal. So in the cases where my engine disagrees with classical Dung, the disagreement is in service of finer-grained feedback for the student, not against it.

5.4 Convergence of the Fixed Point

The acceptability score is defined as the fixed point of a synchronous relaxation. A reasonable question is whether the iteration converges in practice and how fast. I measured this empirically on all sixty scenarios. The harness records the number of iterations until $\max_a |s^{(t+1)}(a) - s^{(t)}(a)| < 10^{-6}$.

Table 5.3: Number of iterations until score-vector stability on each of the sixty scenarios. The iteration ceiling in the deployed engine is ten; no scenario approaches that bound.

Iterations until $\max_a \Delta s < 10^{-6}$	Count
1	11 (18.3%)
2	27 (45.0%)
3	16 (26.7%)
4	2 (3.3%)
5	1 (1.7%)
6	1 (1.7%)
7	1 (1.7%)
8	1 (1.7%)
Median	2
Mean	2.45
Maximum (observed)	8

The median scenario converges in two iterations, the mean in 2.45, and the worst case observed across the benchmark is eight. The deployed engine carries a safety ceiling of ten iterations with an early-termination check. So the bound is never reached in the benchmark and the iteration is empirically fast enough for interactive use. A formal proof of fixed-point existence and uniqueness for the bipolar gradual operator is a known result for closely related semantics [?, ?]; the deployed code relies on the empirical observation rather than on the formal proof.

5.5 Ablation Study

The same sixty scenarios re-run with each engine feature disabled produces Table 5.4. The harness in `tools/run_ablation.py` re-evaluates the root’s verdict and final score under four ablations. Each ablation isolates the contribution of one part of the engine.

Table 5.4: Ablation study: effect on root-claim verdict when each engine feature is disabled in turn. Computed on the same sixty scenarios as Table 5.1.

Disabled component	Verdicts changed	Mean $ \Delta s $
Support relation $\mathcal{R}_{\text{sup}}$	16/60 (26.7%)	0.206
Value-tag multipliers μ	2/60 (3.3%)	0.024
Per-argument weights w	5/60 (8.3%)	0.074
Gradual semantics (Dung grounded instead)	25/60 (41.7%)	0.525

Two findings stand out. Disabling the support relation flips 26.7% of all verdicts, with a mean per-node score perturbation of 0.206. So bipolarity is not decorative: it carries roughly a quarter of the engine’s verdict signal in mixed-shape graphs. Replacing the gradual semantics with classical Dung’s grounded extension flips 41.7% of verdicts with a mean perturbation of 0.525, which is the strongest evidence that my engine is genuinely different from a classical baseline rather than a wrapper around one. Value tags and weights individually flip very few verdicts (3.3% and 8.3% respectively), but their effect is largest precisely in the scenarios where the human-expert label was a close call, which is consistent with the per-topology baseline numbers in Section 5.3.2: value tags matter most on STR scenarios where attackers and defenders share weights but differ in tag, and weights matter most on BIP scenarios where supporters and attackers can be asymmetrically heavy.

5.6 Unit Tests

The PyTest suite under `tests/` contains **101 assertions**, all passing. Coverage:

Engine tests (`tests/test_engine.py`). Score bounds across random graphs, agreement with classical Dung on chain and star topologies, value-tag asymmetry direction (Fact-on-Emotion is harder than the reverse), bipolar support behaviour (a supporter never lowers its target’s score), convergence (the relaxation loop terminates within ten iterations), the `diagnose()` helper, dict-roundtrip serialisation, and weight clamping.

MDP tests (`tests/test_hint_mdp.py`). The state space is exactly eighteen states, the action space is four actions, the optimal policy maps every state to a valid action, losing states always pick `STEP_BACK`, high-pressure winning and tied states always pick `VALUE_REFRAME`, every transition distribution sums to one within floating-point tolerance, and the empirical landing-probability table covers every action.

Scenario regression (`tests/test_scenarios.py`). Every one of the sixty scenarios in Section 5.3 runs through the engine producing a valid score in $[0, 1]$ and a valid status in $\{\text{IN}, \text{OUT}\}$. A regression test enforces a hard agreement floor at 80%, so any future engine change that drops the agreement below that threshold breaks the build immediately.

TikZ export (`tests/test_tikz_export.py`). Smoke tests on the figure exporter: empty engine returns an explanatory comment, every node appears in the output, attack and support edges use distinct styles, and special characters in argument text get properly escaped for LaTeX.

The standalone runner `run_tests.py` executes the same suite without requiring PyTest to be installed, which makes the tests portable to environments where pip is not available.

5.7 Behavioural Observations About the AI

I want to know whether the AI’s behaviour stays coherent with my engine’s verdict, since that coherence is the central pedagogical claim of my system. From the sessions I have personally played through, two things have held up.

The user-visible momentum bar always reflects my engine’s verdict, never the LLM’s, by construction. There is no code path in which the LLM’s text overrides the bar. So in any session where the human’s most recent argument is logically airtight, the bar tilts in their favour regardless of how confidently the AI’s reply is phrased. I have not in my own use seen a case where the AI continued the debate textually while my engine had already declared its argument `OUT`. If anything close to that happened, the user-facing display still showed my engine’s verdict, since that is what every score widget reads from.

The `CONCEDE` sentinel is reached often enough that I have seen it land cleanly. When the AI gives up, my system stops asking it for further moves and the victory banner shows. I have not counted concessions across a structured sample, so I cannot quote a rate.

5.8 Performance Impressions

A debate is only educational if it feels conversational. From my own use of the system, the bottleneck is the LLM round trip rather than anything in my code. The engine itself runs effectively instantaneously at the graph sizes a real debate produces (well under a millisecond on graphs of up to thirty nodes by informal stopwatch). The Streamlit script rerun is fast enough that I do not notice it. The Groq inference latency for a text-only counter-argument is short enough that the cosmetic “thinking” animation successfully masks it; the image-grounded variant takes noticeably longer, but image evidence was a less common move in informal play. **I have not run a controlled latency benchmark.** Reporting median latencies across machines and network conditions is listed as future work in Chapter ??.

5.9 Pedagogical Impressions from Informal Use

The classmates who used my system gave loose, paraphrased feedback after their sessions. The themes below are recurring impressions from those conversations. None of this is statistical and none of it should be read as more than illustrative.

The momentum bar gets more attention than the per bubble score. Most of the testers I spoke to said the bar is what they look at to decide whether they are winning. The per-node acceptability percentage was either ignored or only consulted when the bar moved unexpectedly.

The logic map is intimidating but eventually useful. Several testers ignored the expandable graph until they got confused about why a particular argument’s score had dropped. After they opened it once, the graph became their tool for diagnosing what had gone wrong, especially when an opposing argument had slipped in via a support edge they had not noticed.

On demand hints get used. The hint button was used in most sessions I saw. The hint text was not always perfectly targeted, but it gave enough of a strategic angle to be worth pressing. Now that the underlying strategy is chosen by an MDP whose reward is the engine’s verdict (Section 3.5), the hint is at least defensibly aimed. Whether it is empirically better than a naive single-call LLM hint is what the comparison harness is designed to measure once the human ratings come in.

Voice input matters more than I expected, especially in blitz mode. A few testers tried blitz mode and reported that typing felt like the bottleneck once the timer pressure kicked in. The voice button was a relief for them. Whisper’s transcription was accurate enough that nobody mentioned a transcription failure that prevented them from making a move.

The AI conceding was a surprise. The single most cited reaction from testers was the moment the AI gave up. People paraphrased it differently, but the gist was the same: it was the first time the chatbot felt like it could lose fairly. That impression is worth more pedagogically than any specific latency number, and it is the part of my system I am proudest of.

5.10 Cost

I built my system on free tier services on purpose, so a student could use it without a credit card. In all my development and testing I did not exceed Groq’s free tier rate limits, and I never had to provide payment information. I did not record exact API consumption.

5.11 Threats to Validity

Several limitations apply to the claims in this chapter.

The human-expert labels are mine. Every one of the sixty scenarios was labelled by me. A proper evaluation would use multiple independent raters and report inter-rater agreement with a kappa statistic. The rater spreadsheet for that (`tools/make_rater_template.py`) is built but not yet populated. The baseline comparison in Section 5.3.2 partially mitigates this concern: if my labels were biased toward what my engine would output, the gap of roughly twenty percentage points absolute over both baselines would shrink under unbiased relabelling, but it is unlikely to disappear entirely.

No controlled user study. Nothing in this chapter measures actual learning gain. The qualitative themes I report are filtered through my own bias as the author of the system, and through the small set of classmates who happened to be available to test it.

No control condition. I never compared my full system against a stripped-down baseline (a vanilla LLM debate partner, or a binary-only logic engine) in a user study. So the apparent value of features like the momentum bar or the value tags is inferred from the ablation table, not measured against a real alternative in the hands of real students.

Hand-tuned parameters. The value-tag multipliers (1.2/1.1/1.0/0.8), the score thresholds (0.5 for IN/OUT, 0.65 for HIGH pressure, 0.40 for LOSING), the fatigue multipliers in the MDP (0.85 and 0.65), and the transition probabilities are all hand-chosen. None of them is learned from data. The codebase has a stub function `update_transition_model_from_logs()` for empirical re-estimation of the MDP transitions, but it requires logged student trajectories I do not yet have.

LLM nondeterminism. The Groq API does not guarantee bit-exact reproducibility. So even if a future evaluation re-runs the same scripted debate, the AI’s specific replies will vary. A controlled study would need to average over enough replicates to absorb that variance.

Single environment. Everything I observed was on my own machine, with my own network, in my own city. Performance and reliability on different hardware or in less stable networks are unknown.

Per-topology power. The chi-squared test in Section 5.3.1 reports that the apparent per-topology variation is not statistically significant at $n = 15$ per topology. A larger benchmark of perhaps 40 to 60 scenarios per topology would be needed to confirm or rule out a topology effect.

5.12 AI-versus-AI Tournament

Result. The engine produced a stable verdict on 10 of 12 end-to-end LLM-versus-LLM debates run across six topics with starter swap, holding the same IN/OUT outcome under both provider orderings on every topic. The two OUT verdicts both occurred on a deliberately contrarian motion (*Human teachers should be replaced by AI tutors*), and they occurred regardless of which provider was assigned the proponent role. On every other topic the proponent’s main claim survived under both starter orderings, but with score margins ranging from 0.59 (narrow) to 1.00 (saturated), so the bipolar gradual rule with $\kappa = 0.5$ (Section 3.4.4) preserved a meaningful gradient even on debates where the verdict was the same.

The tournament consisted of six topics covering educational policy (*Schools should ban smartphones in classrooms*, *Universities should replace exams with projects*), social ethics (*Animal testing should be banned*, *Fast fashion should be heavily regulated*), public-health regulation (*Cigarettes should be banned completely*), and one deliberately contrarian proposition designed to test stance bias (*Human teachers should be replaced by AI tutors*). Each topic was debated twice with the two LLM providers swapping the starter role, for a total of twelve debates of eight turns each and ninety-six engine-judged moves. Side A always played proponent, Side B opponent. The two providers were **Groq** serving `llama-3.3-70b-versatile` [?, ?] and **OpenRouter** serving `meta-llama/llama-3.3-70b-instruct`, **both used on free tiers at zero total API cost**, so the entire tournament is reproducible by any reader willing to register for two free API keys. After every move my engine ran `evaluate_semantics()` and recorded the resulting score and status. The verdict on the proponent’s main claim (`Msg_1`) is the reported outcome.

Table 5.5: Outcome of `Msg_1` (the proponent’s main claim) under each starter ordering across the six tournament topics. *Groq-A* means Groq played the proponent; *OR-A* means OpenRouter played the proponent. Score is the engine’s final acceptability for `Msg_1`.

Topic	Groq-A: status (score)	OR-A: status (score)
Schools should ban smartphones	IN (0.91)	IN (0.59)
Universities replace exams with projects	IN (1.00)	IN (0.65)
Replace human teachers with AI tutors	OUT (0.38)	OUT (0.23)
Fast fashion heavily regulated	IN (0.88)	IN (1.00)
Animal testing banned	IN (0.88)	IN (0.67)
Cigarettes banned completely	IN (1.00)	IN (1.00)

Table 5.5 shows that on every topic the binary verdict (IN vs OUT) holds under both starter orderings even though the absolute scores and the resulting graph topologies differ. The smartphones topic, for instance, lands at 0.91 when Groq opens and 0.59 when OpenRouter opens (Figure 5.1). The verdict is the same but the second debate hovers just above the 0.5 threshold. This is the soft-cap rule from Section 3.4.4 doing its intended job: the engine reports *how comfortably* the proponent won, not just *that* the

proponent won. Under the original $\kappa = 1$ formula both runs would have saturated at 1.0 and this distinction would have been lost.

(a) Groq starts as Side A. Verdict: IN at 0.91(b) OpenRouter starts as Side A. Verdict: IN at 0.59.

Figure 5.1: *Schools should ban smartphones in classrooms*. Final argument graphs under both starter orderings. Green nodes carry a Groq move; blue nodes carry an OpenRouter move. Solid red arrows are attacks; dashed blue arrows are supports. Same verdict, different margins to threshold; illustrates the $\kappa = 0.5$ gradient.

The contrarian *replace teachers with AI* topic is the only one in the tournament where the proponent’s claim was defeated, and it was defeated under both starter orderings (0.38 and 0.23, Figure ??). Both providers, when assigned the proponent role, generated arguments that the symbolic engine then judged unable to withstand the opponent’s attacks. This is an honest empirical observation about stance bias in the underlying LLMs: the engine does not see the LLMs trying noticeably harder when assigned a less popular position. It sees the same general pattern of moves and judges the result by the same formula. The verdict structure is therefore informative about *what positions LLMs are willing to defend with their full rhetorical strength*, not just about who is arguing better on a given turn.

At the opposite end of the spectrum the cigarettes-banned topic produces a fully saturated 1.00 verdict in both orderings (Figure ??). I read this as a near-consensus topic in the LLMs’ training data: neither provider can muster a sustained attack against the proponent’s claim regardless of who opens. Note that under the conservative $\kappa = 0.5$ rule, reaching the cap requires the supporter contribution to dominate the attacker contribution by a factor of two. That this happens consistently is evidence that both LLMs treat the proposition as essentially uncontested rather than that they failed to attack at all.

(a) Groq starts as Side A. Verdict: **OUT** at 0.38. (b) OpenRouter starts as Side A. Verdict: **OUT** at 0.23.

Figure 5.2: *Human teachers should be replaced by AI tutors*. The only topic in the tournament where the proponent’s main claim was defeated, and the only one where the defeat held regardless of starter.

Across the ninety-six individual moves the LLMs picked the four value tags non-uniformly, in a pattern that tracks topic genre. Educational topics produced Ethics and Logic dominance, social-ethics topics produced the first Fact tags (likely statistics retrieval), and the replace-teachers debate was the most Logic-heavy of all six. The tags map onto recognisable rhetorical modes even though the LLMs had no scoring incentive to pick honestly. Every one of the twelve debates contains at least one **SUPPORT** edge, with roughly one move in five being a support rather than an attack despite the LLM being free to pick either. The bipolar machinery is therefore being exercised in practice, not just in theory.

These twelve debates are **not** a statistical evaluation; they are a qualitative demonstration that the system runs end to end and produces sensible verdicts on real LLM output. The findings about LLM stance bias on the *replace teachers* topic and about value-tag distribution shifting with genre are observations from a sample of twelve, not significance-tested claims. A controlled study would re-run each topic with at least thirty different random-temperature replicates and report variance bars on the final score; that study is listed as future work in Chapter ???. The remaining nine final argument graphs (the other side of the swap for the three topics covered above, plus the three topics not covered in the main text) are available in Appendix ??.

(a) Groq starts as Side A. Verdict: IN at 1.00 (b) OpenRouter starts as Side A. Verdict: IN at 1.00.

Figure 5.3: *Cigarettes should be banned completely*. Both runs saturate at 1.00, the only topic in the tournament to do so. Evidence of near-consensus in the underlying LLMs.

5.13 Summary

What this chapter establishes, with quantitative evidence: my engine agrees with my human-expert labels on 85% of sixty hand-crafted argumentation scenarios (95% Wilson CI [73.9%, 91.9%], $p < 10^{-7}$ against chance), with the 15% disagreement following three documented patterns that reflect properties of bipolar gradual semantics rather than bugs. On the same scenarios my engine outperforms the published h-categorizer by 21.7 percentage points absolute and the classical Dung grounded extension by 18.3 percentage points, with the largest gap appearing on bipolar scenarios that the classical baseline structurally cannot represent. Disabling the support relation flips 26.7% of verdicts, and replacing the gradual semantics with classical grounded flips 41.7%, so both contributions of my engine over the classical baseline are non-trivial. The fixed-point iteration converges in two to three iterations on the median scenario and never exceeds eight on the benchmark. A PyTest suite of 101 assertions guards every claim in this chapter against regression.

What this chapter does not establish: I have not measured learning outcomes, I have not run a controlled user study, and I have not collected the human ratings needed to compare the MDP-guided hint against the naive single-call baseline. These are the highest-priority follow-ups, and Chapter ?? discusses what each one would look like.

5.14 Reproducibility Appendix

Every number in this chapter can be re-derived from the codebase. The exact commands are listed below for the benefit of any future reader who wants to verify the claims.

Source tree. All scripts live under `tools/` in the project root, and produce CSV and LaTeX-table outputs under `tools/results/`.

Re-running the sixty-scenario audit.

```
cd code_rewritten
python tools/run_evaluation.py
```

produces `tools/results/evaluation_results.csv` (per-scenario verdict, score, iterations, agreement flag), `evaluation_summary.txt` (totals and disagreement detail), and `evaluation_table.tex` (LaTeX source for Table 5.1).

Re-running the baseline comparison. The baseline comparison in Table 5.2 is produced by a small additional script that wraps the same scenario set and runs the pure h-categorizer and the classical Dung grounded labelling. The h-categorizer is the fixed point of $s(a) = 1/(1 + \sum_{b:b \rightarrow a} s(b))$; the grounded labelling is the iterative IN/OUT/UNDEC procedure of Section 2 of [?]. Both implementations are deterministic and seed-free.

Re-running the ablation study.

```
python tools/run_ablation.py
```

produces `tools/results/ablation_results.csv` (per-scenario verdict and score under each ablation), `ablation_summary.txt` (the totals), and `ablation_table.tex` (LaTeX source for Table 5.4).

Re-running the unit tests.

```
python -m pytest tests/
```

or, if PyTest is not installed,

```
python run_tests.py
```

The standalone runner is functionally equivalent and prints one line per assertion.

Convergence statistics. The iteration counts in Table 5.3 are reported in the `iters` column of `evaluation_results.csv`. They are produced by the same single command as the sixty-scenario audit.

Statistical analysis. The Wilson confidence intervals, the binomial test against chance, and the chi-squared test for topology independence are computed from the `agreement` column of `evaluation_results.csv` with standard formulas (Wilson 1927, two-sided exact binomial, Pearson chi-squared with continuity correction). No special package is required beyond the standard library.

Determinism. The engine, the baseline evaluators, the ablation harness, and the unit test suite are all deterministic. Re-running any of the scripts on the same scenarios will produce bit-exact identical CSV outputs. The only non-deterministic part of the system is the LLM, which is exercised by the AI layer but not by any number in this chapter.

Chapter 6

Conclusion and Future Work

6.1 Recap

I set out in this thesis to address a specific gap in the landscape of educational AI. Generative language models can hold a conversation about almost anything, but I find they suffer from rhetorical rigidity [?] and tend to claim victory regardless of whether the underlying logic actually supports them [?]. Formal argumentation frameworks are the opposite story. They can decide who is winning a debate with mathematical certainty [?], but they have no way to engage a learner in natural conversation. My Logic Advocate Tutor is my attempt to close that gap by binding a generative LLM to a deterministic argumentation engine in a way that lets the model do the writing while the engine does the judging.

In Chapter 1 I set up the problem of the “illusion of competence” in AI tutors and motivated the project as a synthesis of two unequal halves of the conversational stack. In Chapter 2 I surveyed the theory. Dung’s abstract argumentation [?], the bipolar extensions of Cayrol and Lagasque-Schiex [?], the gradual semantics of [?] and [?], the value based audience model of Bench-Capon [?], and the formal dialogue games of Prakken [?], alongside more recent results on LLM behaviour in debate and on automatic hint generation [?]. In Chapter 3 I described my four layer architecture and gave the formal definition of my Bipolar Gradual engine. In Chapter 4 I translated the design into Python, paying attention to my prompt engineering protocol that disciplines the LLM and to my polled JSON multiplayer subsystem. In Chapter 5 I reported what works and what does not, drawing on canned debate runs, ongoing personal observation, and informal feedback from classmates.

6.2 Contributions

My work makes five concrete contributions, and each one corresponds to a specific section of my implementation.

A custom Bipolar Gradual engine. In my implementation in `logic_engine.py` (Section 4.2) I packed four argumentation theoretic ideas (bipolarity, gradual semantics, value based weighting, and preference based normalisation) into a single 60 line evaluator. My engine reduces correctly to a thresholded grounded extension on classical inputs, and produces well behaved continuous scores on bipolar, weighted, and value tagged inputs, as I documented in Sections 5.3 and 3.4.5.

A protocol for grounding the LLM. My pipe separated response protocol in `ai_agent.py` and the surrounding context injection pattern (Section 4.3.2) produce an AI opponent whose textual confidence cannot diverge from the engine’s verdict. In every session I observed, the user facing momentum bar agreed with the engine’s verdict rather than the AI’s text, including the cases where the engine declared the AI’s argument defeated and the AI emitted a clean **CONCEDE**. As far as I have been able to find, this is the first deployed implementation of a context injected, math anchored LLM debate partner aimed at undergraduate education.

An on demand hint system. My hint generator in `ai_agent.py` (Section 4.3) operationalises the principle that hints in a logic tutor should be on demand and non evaluative [?]. My hint button does not consume the learner’s turn and produces a single sentence strategic suggestion targeted at the most recent enemy argument. Most of the testers I spoke to used it, and a couple of them leaned on it heavily. The pedagogical value of this feature was a consistent thread in the informal feedback I collected.

Interactive logic visualisation. My combination of the Graphviz rendered logic map and the inline HTML momentum bar (Section 4.4) gives learners two redundant readings of the same engine output, one structural and one summary. The bar was the more used widget. The map was the more explanatory one. Together they let learners ask not only “am I winning?” but “why am I winning?”. That second question is one classical Dung style binary tutors are not equipped to answer at all.

A multimodal, classroom deployable architecture. My Streamlit application, my ngrok tunnel [?], and my QR code joining flow [?] let two students on different networks share a debate session in a few seconds with no installation. Voice input via Whisper [?] and image evidence via the LLaMA-3.2 Vision model [?] extend the same protocol to non text modalities. My whole system runs inside Groq’s [?] free tier, which means the financial cost of classroom adoption is zero.

6.3 Limitations

My system has clear limitations I want to be upfront about. Each one would need to be addressed before the tutor could be deployed at scale.

No automatic argument structure extraction. I require the learner to choose explicitly between *Attack* and *Support*, point at a target, and assign a value tag for every move. I designed it that way on purpose, because it pushes the learner to think structurally. But it also means a free text essay, of the kind analysed by Wachsmuth and Al-Khatib [?], cannot be ingested into my system. A future version could pair my engine with an argument mining preprocessor.

Static value tag multipliers. I baked the constants 1.2/1.1/1.0/0.8 into the source. I picked them by inspection (Section 3.4) and never learned them from data. A future version should expose them as a per classroom or per debate setting, or even better, learn them from tournament data.

Limited validation. My evaluation in Chapter 5 is informal. A handful of classmates, no control group, no learning outcome measurements, and a single demographic of computer science undergraduates at one institution. The findings are best read as a sanity check rather than as evidence that my system improves learning.

LLM nondeterminism. The Groq endpoint does not guarantee bit exact reproducibility. My engine’s verdict is deterministic, but the AI’s prose around that verdict is not. I find that fine for tutoring but problematic for any future research use that needs reproducible transcripts.

Polling based multiplayer. My lobby store is a JSON file polled at three second intervals (Section 4.5). I find that fine for two party debates between cooperative players, but it does not scale to large classrooms or to adversarial settings where one player might try to hammer the polling interval.

No undercut relation in production. My legacy class `Argument.txt` prototype (Section 3.4.6) had Pollock style *undercut* attacks, the kind that break the inferential bridge between premise and claim instead of the claim itself. My deployed engine collapses undercuts into ordinary attacks. Restoring the distinction would let my system express a richer set of dialectical moves.

6.4 Future Work

The limitations above suggest a concrete forward roadmap.

Argument mining as a front end. I could replace or augment the explicit target UI with a sentence level argument mining pipeline, which would let my system accept short essays, transcripts of class discussions, or news articles. The output of an argument miner is a graph of claims and premises, which is exactly the input my engine expects. Plugging in an argument miner trained on essays [?] would dramatically widen the kinds of input my tutor can grade.

Learned value tag multipliers. With a moderate amount of tournament data (maybe 200 multi move debates between human pairs), I could fit the four value tag multipliers, or even an off diagonal matrix instead of a diagonal one, so the engine’s predicted winner agrees with the human judged winner. That would replace my hand tuned constants with empirically grounded ones and let an educator tune the audience to their classroom.

Restoring undercut attacks. I could reintroduce Pollock style undercut attacks to make my framework expressive enough to model cases where a learner challenges the inference rather than the conclusion. The data structure already exists in my prototype. My deployed engine would need a third relation $\mathcal{R}_{\text{und}}$ that scales the strength of an attack edge rather than the score of a node, and a corresponding UI for selecting an edge as a target.

A proper user study. I could run a real between subjects study comparing three conditions. (a) my full Logic Advocate Tutor, (b) a vanilla LLM debate partner with no engine, and (c) a classical binary Dung tutor with no LLM. Learning gain would be measured on a short pre and post argument quality test scored against the rubric of Wachsmuth et al. [?]. Sample sizes of 30 per arm would be enough to detect a moderate effect size difference. This is the single most valuable next study, and it was infeasible only because of the time budget of a one semester thesis.

Persistent learner profiles. At the moment my system has no model of the learner across sessions. A simple addition would be to record per learner statistics about which value tags they over use, which kinds of attacks they fail to defend against, and which kinds of arguments tend to be at the root of their winning debates. Those statistics could feed a meta tutor that nudges learners toward the styles they are weakest at, in the spirit of VanLehn’s classification of model tracing tutoring [?].

Real time multiplayer scaling. I could replace my polled JSON file with a small WebSocket server, which would drop the move to mirror latency to milliseconds and would also kill the residual lobby bug observed in the evaluation. That is the highest value engineering improvement for a real classroom deployment, and it would not require any change to my formal logic.

Multilingual support. My current prompt is English only. The Whisper transcription [?] already supports Arabic, French, and German out of the box, but my system prompt and value tag labels would need to be localised before my system could be used outside an English medium classroom.

6.5 Closing Thoughts

The deepest lesson I take away from this project is small. The boundary between conversational AI and formal logic is best policed by code, not by prose. Asking an LLM to “be honest about whether it is losing” is asking the wrong agent the wrong question. Making the LLM *unable to lie*, by depriving it of the privilege of computing the verdict, turns out to be much easier and much more reliable. Once that boundary is in place, the LLM gets to do the thing it is genuinely best at (producing readable, persuasive prose), and the formal framework gets to do the thing *it* is best at, which is keeping score on the structure of a disagreement. My Logic Advocate Tutor is one specific instance of that division of labour, but I think the principle generalises. I suspect that future tools combining generative language with verifiable reasoning are going to look, structurally, very similar.

I offer the work submitted with this thesis both as an artefact (a deployable, classroom usable debate tutor) and as a small piece of evidence that the right way to build educational AI in the era of large language models is to keep the model and the rules in two different rooms, and to let the rules win whenever they disagree.

Bibliography

- [1] P. A. Aoyagui, K. Stemmler, S. A. Ferguson, Y.-H. Kim, and A. Kuzminykh, “A matter of perspective(s): Contrasting human and llm argumentation in subjective decision-making on subtle sexism,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ACM, 2025.
- [2] H. Prakken, “Coherence and flexibility in dialogue games for argumentation,” *Journal of Logic and Computation*, vol. 15, no. 6, pp. 1009–1040, 2005.
- [3] P. M. Dung, “On the acceptability of arguments and its fundamental role in non-monotonic reasoning, logic programming and n-person games,” *Artificial Intelligence*, vol. 77, no. 2, pp. 321–357, 1995.
- [4] H. B. Ku, J. Shin, H. J. Lee, S. Na, and I. Jeon, “Multi-agent llm debate unveils the premise left unsaid,” in *Proceedings of the 12th Argument Mining Workshop*, pp. 58–73, Association for Computational Linguistics, 2025.
- [5] J. C. Stamper, M. Eagle, T. Barnes, and M. Croy, “Experimental evaluation of automatic hint generation for a logic tutor,” in *Proceedings of the International Conference on Artificial Intelligence in Education (AIED)*, pp. 345–352, 2011.
- [6] P. Baroni, M. Caminada, and M. Giacomin, “Abstract argumentation frameworks and their semantics,” *Handbook of Formal Argumentation*, pp. 159–236, 2018.
- [7] C. Cayrol and M.-C. Lagasquie-Schiex, “On the acceptability of arguments in bipolar argumentation frameworks,” in *Symbolic and Quantitative Approaches to Reasoning with Uncertainty (ECSQARU 2005)*, vol. 3571 of *Lecture Notes in Computer Science*, pp. 378–389, Springer, 2005.
- [8] L. Amgoud, J. Ben-Naim, D. Doder, and S. Vesic, “Acceptability semantics for weighted argumentation frameworks,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 56–62, 2017.
- [9] P. Besnard and A. Hunter, “A logic-based theory of deductive arguments,” *Artificial Intelligence*, vol. 128, no. 1-2, pp. 203–235, 2001.
- [10] T. J. M. Bench-Capon, “Persuasion in practical argument using value-based argumentation frameworks,” *Journal of Logic and Computation*, vol. 13, no. 3, pp. 429–448, 2003.

- [11] F. Castagna, N. Kökciyan, I. Sassoön, S. Parsons, and E. Sklar, “Computational argumentation-based chatbots: a survey,” *Journal of Artificial Intelligence Research*, vol. 80, pp. 1271–1310, 2024.
- [12] C. Rapanta and F. Macagno, “Argumentation methods in educational contexts: Introduction to the special issue,” *International Journal of Educational Research*, vol. 79, pp. 142–149, 2016.
- [13] C. Cayrol and M.-C. Lagasquie-Schiex, “Graduality in argumentation,” *Journal of Artificial Intelligence Research*, vol. 23, pp. 245–297, 2005.
- [14] H. Wachsmuth, K. Al-Khatib, and B. Stein, “Using argument mining to assess the argumentation quality of essays,” in *Proceedings of COLING 2016*, pp. 1680–1691, 2016.
- [15] S. E. Toulmin, “The uses of argument,” 2003.
- [16] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull, “Graphviz—open source graph drawing tools,” in *Graph Drawing: 9th International Symposium*, pp. 483–484, Springer, 2002.
- [17] Streamlit Inc., “Streamlit: The fastest way to build and share data apps.” <https://streamlit.io>, 2024.
- [18] L. Amgoud and C. Cayrol, “A reasoning model based on the production of acceptable arguments,” *Annals of Mathematics and Artificial Intelligence*, vol. 34, no. 1-3, pp. 197–215, 2002.
- [19] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [20] Meta AI, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [21] C. Rapanta and F. Macagno, “Argumentation as a teaching and learning tool,” in *Argumentation in Education*, Springer, 2019.
- [22] N.-T. Le, “How do technology-enhanced learning environments support students’ argumentation?,” *Frontiers in Education*, vol. 4, 2019.
- [23] Groq Inc., “Groq api documentation.” <https://console.groq.com/docs>, 2024.
- [24] ngrok Inc., “ngrok documentation.” <https://ngrok.com/docs>, 2024.
- [25] P. Lincoln, “qrcode: Qr code image generator.” <https://pypi.org/project/qrcode/>, 2024.

- [26] H. Wachsmuth, N. Naderi, Y. Hou, Y. Bilu, V. Prabhakaran, T. A. Thijm, G. Hirst, and B. Stein, “Computational argumentation quality assessment in natural language,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pp. 176–187, 2017.
- [27] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.